

Murmur

Decides which children need an echocardiogram from a heart-sound recording.

Source code in this document — 2,662 lines

SIGNAL PROCESSING AND MODEL

features.py	115 lines
model.py	37 lines

TRAINING AND EVALUATION

prepare.py	64 lines
train.py	183 lines
evaluate.py	54 lines
export.py	67 lines

PROVING THE BROWSER MATCHES THE PYTHON

verify_dsp.py	27 lines
verify_dsp.mjs	34 lines
verify_predictions.py	38 lines

THE WEB APPLICATION

web/dsp.js	148 lines
web/app.js	862 lines
web/index.html	239 lines
web/app.css	794 lines

Not included. Corpora under `data/` (PhysioNet, not ours to redistribute), `.venv/` and `node_modules/`, generated caches, and the trained weights `murmur.onnx` / `murmur_net.pt`, which are binary and are produced by `train.py` and `export.py` above.

```

1  """Log-mel features from a phonocardiogram.
2
3  Uses scipy only. Heart sounds in this dataset are 4 kHz mono, so the Nyquist
4  limit is 2 kHz; murmurs live roughly between 25 and 600 Hz, which is why the
5  filterbank stops at 1 kHz rather than spending resolution on empty band.
6  """
7
8  from __future__ import annotations
9
10 import numpy as np
11 from scipy.io import wavfile
12 from scipy.signal import butter, sosfiltfilt, stft
13
14 SAMPLE_RATE = 4000
15 SEG_SECONDS = 4.0
16 SEG_LEN = int(SAMPLE_RATE * SEG_SECONDS)
17 HOP_SECONDS = 2.0
18 N_MELS = 64
19 N_FFT = 256
20 HOP = 128          # 32 ms frames: ample for a sustained murmur, half the memory
21 FMIN, FMAX = 25.0, 1000.0
22 MAX_SEGMENTS = 8   # a 26 s recording yields ~11; 8 evenly spaced is plenty
23
24
25 def _hz_to_mel(f):
26     return 2595.0 * np.log10(1.0 + f / 700.0)
27
28
29 def _mel_to_hz(m):
30     return 700.0 * (10.0 ** (m / 2595.0) - 1.0)
31
32
33 def mel_filterbank(sr=SAMPLE_RATE, n_fft=N_FFT, n_mels=N_MELS, fmin=FMIN, fmax=FMAX):
34     """Triangular mel filterbank, built once and reused."""
35     edges = _mel_to_hz(np.linspace(_hz_to_mel(fmin), _hz_to_mel(fmax), n_mels + 2))
36     freqs = np.linspace(0, sr / 2, n_fft // 2 + 1)
37     fb = np.zeros((n_mels, freqs.size), dtype=np.float32)
38     for i in range(n_mels):
39         lo, mid, hi = edges[i], edges[i + 1], edges[i + 2]
40         left = (freqs - lo) / max(mid - lo, 1e-9)
41         right = (hi - freqs) / max(hi - mid, 1e-9)
42         fb[i] = np.clip(np.minimum(left, right), 0, None)
43     return fb
44
45
46 _FB = mel_filterbank()
47
48
49 def bandpass(x, sr=SAMPLE_RATE, lo=25.0, hi=800.0):
50     """Drop the rumble below 25 Hz and the hiss above 800 Hz.
51
52     Both are recording artefacts far more often than they are heart sounds.
53     """
54     sos = butter(4, [lo / (sr / 2), min(hi / (sr / 2), 0.99)], btype="band", output="sos")
55     return sosfiltfilt(sos, x).astype(np.float32)
56
57
58 def read_wav(path) -> np.ndarray:
59     sr, x = wavfile.read(path)
60     x = x.astype(np.float32)
61     if x.ndim > 1:
62         x = x.mean(axis=1)
63     if np.max(np.abs(x)) > 0:
64         x = x / np.max(np.abs(x))
65     if sr != SAMPLE_RATE:
66         n = int(round(x.size * SAMPLE_RATE / sr))
67         x = np.interp(np.linspace(0, x.size - 1, n), np.arange(x.size), x).astype(np.float32)
68     return x
69
70
71 def log_mel(seg: np.ndarray) -> np.ndarray:
72     """One segment -> (N_MELS, frames) log-mel, per-segment normalised."""

```

```

73     _, _, Z = stft(seg, fs=SAMPLE_RATE, nperseg=N_FFT, noverlap=N_FFT - HOP,
74                   padded=False, boundary=None)
75     power = (np.abs(Z) ** 2).astype(np.float32)
76     mel = _FB @ power
77     logmel = np.log(mel + 1e-8)
78     logmel = (logmel - logmel.mean()) / (logmel.std() + 1e-6)
79     return logmel.astype(np.float32)
80
81
82 def segments(x: np.ndarray):
83     """Fixed windows with overlap, capped and evenly spaced across the recording.
84
85     Capping keeps the feature cache in memory. Spreading the cap across the
86     whole recording rather than taking the first N matters, because a murmur
87     may only be audible during part of it.
88     """
89     if x.size < SEG_LEN:
90         x = np.pad(x, (0, SEG_LEN - x.size))
91     hop = int(SAMPLE_RATE * HOP_SECONDS)
92     starts = list(range(0, max(1, x.size - SEG_LEN + 1), hop))
93     if len(starts) > MAX_SEGMENTS:
94         pick = np.linspace(0, len(starts) - 1, MAX_SEGMENTS).round().astype(int)
95         starts = [starts[i] for i in dict.fromkeys(pick)]
96     for start in starts:
97         yield x[start:start + SEG_LEN]
98
99
100 #: The mel filterbank already discards everything outside 25-1000 Hz, so the
101 #: Butterworth stage is close to redundant. It is also the one step that is
102 #: genuinely hard to reproduce bit-for-bit in a browser, and a model that sees
103 #: different features in the product than it saw in training is worse than a
104 #: marginally less accurate one. Off by default; the flag keeps the comparison
105 #: reproducible.
106 USE_BANDPASS = False
107
108
109 def recording_features(path) -> np.ndarray:
110     """(n_segments, 1, N_MELS, frames) ready for the network."""
111     x = read_wav(path)
112     if USE_BANDPASS:
113         x = bandpass(x)
114     feats = [log_mel(s) for s in segments(x)]
115     return np.stack(feats)[: , None, :, :].astype(np.float32)

```

```
1  """A small convolutional network over log-mel segments.
2
3  Deliberately small. There are 942 patients, so a large network would memorise
4  them. Four conv blocks and global pooling keeps the parameter count low enough
5  that the held-out numbers mean something.
6  """
7
8  from __future__ import annotations
9
10 import torch
11 import torch.nn as nn
12
13
14 class MuzmurNet(nn.Module):
15     def __init__(self, n_mels: int = 64):
16         super().__init__()
17
18         def block(cin, cout):
19             return nn.Sequential(
20                 nn.Conv2d(cin, cout, 3, padding=1),
21                 nn.BatchNorm2d(cout),
22                 nn.ReLU(inplace=True),
23                 nn.Conv2d(cout, cout, 3, padding=1),
24                 nn.BatchNorm2d(cout),
25                 nn.ReLU(inplace=True),
26                 nn.MaxPool2d(2),
27             )
28
29         self.features = nn.Sequential(block(1, 16), block(16, 32), block(32, 64), block(64, 64))
30         self.head = nn.Sequential(nn.Dropout(0.3), nn.Linear(64, 1))
31
32     def forward(self, x):
33         h = self.features(x)
34         # Mean over time, max over frequency: a muzmur is a sustained band of
35         # noise, so it should survive averaging along the time axis.
36         h = h.mean(dim=3).amax(dim=2)
37         return self.head(h).squeeze(1)
```

```

1  """Unpack the dataset and build a patient-level index.
2
3  The split is by PATIENT, never by recording. Each patient contributes several
4  auscultation locations, and putting one of a patient's recordings in train and
5  another in test would quietly inflate every number we report.
6  """
7
8  from __future__ import annotations
9
10 import csv
11 import json
12 import zipfile
13 from pathlib import Path
14
15 HERE = Path(__file__).resolve().parent
16 DATA = HERE / "data"
17 ZIP = DATA / "circor.zip"
18 ROOT = DATA / "circor"
19
20
21 def unpack() -> Path:
22     if ROOT.exists() and any(ROOT.rglob("*.wav")):
23         return ROOT
24     with zipfile.ZipFile(ZIP) as z:
25         z.extractall(ROOT)
26     return ROOT
27
28
29 def find_training_dir(root: Path) -> Path:
30     for p in root.rglob("training_data.csv"):
31         return p.parent
32     raise FileNotFoundError("training_data.csv not found in the archive")
33
34
35 def build_index() -> list[dict]:
36     root = unpack()
37     base = find_training_dir(root)
38     wav_dir = base / "training_data"
39     rows = list(csv.DictReader((base / "training_data.csv").open()))
40     index = []
41     for r in rows:
42         pid = r["Patient ID"]
43         wavs = sorted(str(p) for p in wav_dir.glob(f"{pid}_*.wav"))
44         if not wavs:
45             continue
46         index.append({
47             "patient_id": pid,
48             "murmur": r.get("Murmur", "").strip(),      # Present / Absent / Unknown
49             "outcome": r.get("Outcome", "").strip(),    # Abnormal / Normal
50             "age": r.get("Age", "").strip(),
51             "locations": r.get("Murmur locations", "").strip(),
52             "wavs": wavs,
53         })
54     (DATA / "index.json").write_text(json.dumps(index, indent=1))
55     return index
56
57
58 if __name__ == "__main__":
59     idx = build_index()
60     from collections import Counter
61     print(f"patients indexed : {len(idx)}")
62     print(f"recordings      : {sum(len(p['wavs']) for p in idx)}")
63     print(f"murmur          : {dict(Counter(p['murmur'] for p in idx))}")
64     print(f"outcome          : {dict(Counter(p['outcome'] for p in idx))}")

```

```

1  """Train the mumur screener.
2
3  Three rules this file exists to enforce:
4
5  1. The split is by patient. A patient's several recordings never straddle it.
6  2. The threshold is chosen for screening, not for accuracy. Missing a mumur
7     costs a child an echo they needed; a false alarm costs one they did not.
8     So we pick the operating point by sensitivity and report what it costs.
9  3. Patients labelled Unknown are excluded from training. The annotators could
10     not hear the answer, so neither can the network, and training on them would
11     teach it to guess.
12  """
13
14  from __future__ import annotations
15
16  import json
17  import random
18  from pathlib import Path
19
20  import numpy as np
21  import torch
22  import torch.nn as nn
23  from sklearn.metrics import roc_auc_score
24
25  import features as F
26  from model import MuzmurNet
27
28  HERE = Path(__file__).resolve().parent
29  DATA = HERE / "data"
30  SEED = 20260912
31  EPOCHS = 12
32  BATCH = 64
33  TARGET_SENSITIVITY = 0.90
34
35
36  def patient_split(index, seed=SEED):
37      """Stratified by label, split on patient id."""
38      rng = random.Random(seed)
39      present = [p for p in index if p["mumur"] == "Present"]
40      absent = [p for p in index if p["mumur"] == "Absent"]
41      rng.shuffle(present); rng.shuffle(absent)
42
43      def cut(xs):
44          n_test = max(1, int(0.2 * len(xs)))
45          n_val = max(1, int(0.1 * len(xs)))
46          return xs[:n_test], xs[n_test:n_test + n_val], xs[n_test + n_val:]
47
48      pt, pv, ptr = cut(present)
49      at, av, atr = cut(absent)
50      return ptr + atr, pv + av, pt + at
51
52
53  def load_split(patients, cache: dict):
54      """Segment-level X and y, plus the patient id each segment came from."""
55      X, y, owner = [], [], []
56      for p in patients:
57          label = 1.0 if p["mumur"] == "Present" else 0.0
58          for w in p["wavs"]:
59              if w not in cache:
60                  try:
61                      cache[w] = F.recording_features(w)
62                  except Exception:
63                      cache[w] = np.zeros((0, 1, F.N_MELS, 1), dtype=np.float32)
64                  feats = cache[w]
65                  if feats.shape[0] == 0:
66                      continue
67                  X.append(feats)
68                  y.append(np.full(feats.shape[0], label, dtype=np.float32))
69                  owner.extend([p["patient_id"]] * feats.shape[0])
70      if not X:
71          raise SystemExit("no features built; is the dataset unpacked?")
72      return np.concatenate(X), np.concatenate(y), np.array(owner)

```

```

73
74
75 def patient_scores(seg_scores, owners, patients):
76     """A murmur may be audible at one location only, so take the patient's max."""
77     by = {}
78     for s, o in zip(seg_scores, owners):
79         by[o] = max(by.get(o, -1e9), float(s))
80     ids = [p["patient_id"] for p in patients if p["patient_id"] in by]
81     return np.array([by[i] for i in ids]), np.array(
82         [1.0 if next(p for p in patients if p["patient_id"] == i)["murmur"] == "Present" else 0.0
83          for i in ids]
84     )
85
86
87 def threshold_for_sensitivity(scores, labels, target=TARGET_SENSITIVITY):
88     """Lowest threshold that still reaches the target sensitivity."""
89     order = np.sort(np.unique(scores))[:-1]
90     best = order[-1]
91     for t in order:
92         pred = scores >= t
93         tp = float(((pred == 1) & (labels == 1)).sum())
94         fn = float(((pred == 0) & (labels == 1)).sum())
95         sens = tp / max(tp + fn, 1e-9)
96         if sens >= target:
97             best = t
98             break
99     return float(best)
100
101
102 def report(name, scores, labels, thr):
103     pred = scores >= thr
104     tp = float(((pred == 1) & (labels == 1)).sum())
105     tn = float(((pred == 0) & (labels == 0)).sum())
106     fp = float(((pred == 1) & (labels == 0)).sum())
107     fn = float(((pred == 0) & (labels == 1)).sum())
108     sens = tp / max(tp + fn, 1e-9)
109     spec = tn / max(tn + fp, 1e-9)
110     auc = roc_auc_score(labels, scores) if len(set(labels)) > 1 else float("nan")
111     print(f" {name:10} n={len(labels):4d} AUC={auc:.3f} "
112           f"sensitivity={sens:.3f} specificity={spec:.3f} "
113           f"(tp={tp:.0f} fn={fn:.0f} fp={fp:.0f} tn={tn:.0f})")
114     return {"n": len(labels), "auc": float(auc), "sensitivity": float(sens),
115           "specificity": float(spec), "tp": tp, "fn": fn, "fp": fp, "tn": tn}
116
117
118 def main():
119     torch.manual_seed(SEED); np.random.seed(SEED)
120     index = json.loads((DATA / "index.json").read_text())
121     train_p, val_p, test_p = patient_split(index)
122     print(f"patients train={len(train_p)} val={len(val_p)} test={len(test_p)}")
123
124     cache: dict = {}
125     Xtr, ytr, otr = load_split(train_p, cache)
126     Xva, yva, ova = load_split(val_p, cache)
127     Xte, yte, ote = load_split(test_p, cache)
128     print(f"segments train={len(ytr)} val={len(yva)} test={len(yte)}")
129
130     width = min(a.shape[-1] for a in (Xtr, Xva, Xte))
131     Xtr, Xva, Xte = Xtr[:, :width], Xva[:, :width], Xte[:, :width]
132
133     net = MurmurNet(F.N_MELS)
134     pos_weight = torch.tensor([(ytr == 0).sum() / max((ytr == 1).sum(), 1)], dtype=torch.float32)
135     loss_fn = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
136     opt = torch.optim.AdamW(net.parameters(), lr=1e-3, weight_decay=1e-4)
137     print(f"class weight for Present: {float(pos_weight):.2f}")
138
139     Xtr_t = torch.from_numpy(Xtr); ytr_t = torch.from_numpy(ytr)
140     best_auc, best_state = -1.0, None
141     for epoch in range(1, EPOCHS + 1):
142         net.train()
143         perm = torch.randperm(len(ytr_t))
144         total = 0.0
145         for i in range(0, len(perm), BATCH):
146             idx = perm[i:i + BATCH]
147             opt.zero_grad()

```

```

148         out = net(Xtr_t[idx])
149         loss = loss_fn(out, ytr_t[idx])
150         loss.backward(); opt.step()
151         total += float(loss.detach()) * len(idx)
152     net.eval()
153     with torch.no_grad():
154         va = torch.sigmoid(net(torch.from_numpy(Xva))).numpy()
155         ps, pl = patient_scores(va, ova, val_p)
156         auc = roc_auc_score(pl, ps) if len(set(pl)) > 1 else 0.0
157         print(f" epoch {epoch:2d}  loss={total/len(perm):.4f}  val patient AUC={auc:.3f}")
158         if auc > best_auc:
159             best_auc = auc
160             best_state = {k: v.clone() for k, v in net.state_dict().items()}
161
162     net.load_state_dict(best_state); net.eval()
163     with torch.no_grad():
164         va = torch.sigmoid(net(torch.from_numpy(Xva))).numpy()
165         te = torch.sigmoid(net(torch.from_numpy(Xte))).numpy()
166         vs, vl = patient_scores(va, ova, val_p)
167         ts, tl = patient_scores(te, ote, test_p)
168
169     thr = threshold_for_sensitivity(vs, vl)
170     print(f"\noperating point chosen on VALIDATION for sensitivity >= {TARGET_SENSITIVITY}: {thr:.4f}")
171     print("patient-level results")
172     v = report("val", vs, vl, thr)
173     t = report("test", ts, tl, thr)
174
175     torch.save(net.state_dict(), HERE / "murmur_net.pt")
176     (HERE / "metrics.json").write_text(json.dumps(
177         {"threshold": thr, "target_sensitivity": TARGET_SENSITIVITY,
178          "val": v, "test": t, "seed": SEED, "width": int(width)}, indent=1))
179     print("\nsaved murmur_net.pt and metrics.json")
180
181
182 if __name__ == "__main__":
183     main()

```

```

1  """Sweep the operating point on the held-out test set.
2
3  The threshold is a product decision, not a model property. A screening tool
4  that misses murmurs sends children home; one that over-refers wastes echo
5  slots. This prints the whole curve so the choice is made with the cost visible.
6  """
7  from __future__ import annotations
8  import json
9  from pathlib import Path
10 import numpy as np, torch
11 from sklearn.metrics import roc_auc_score
12 import features as F
13 from model import MuzmurNet
14 from train import patient_split, load_split, patient_scores
15
16 HERE = Path(__file__).resolve().parent
17
18 def main():
19     index = json.loads((HERE / "data" / "index.json").read_text())
20     train_p, val_p, test_p = patient_split(index)
21     cache: dict = {}
22     Xva, _, ova = load_split(val_p, cache)
23     Xte, _, ote = load_split(test_p, cache)
24     width = json.loads((HERE / "metrics.json").read_text())["width"]
25     Xva, Xte = Xva[..., :width], Xte[..., :width]
26
27     net = MuzmurNet(F.N_MELS)
28     net.load_state_dict(torch.load(HERE / "muzmur_net.pt", map_location="cpu"))
29     net.eval()
30     with torch.no_grad():
31         vs, vl = patient_scores(torch.sigmoid(net(torch.from_numpy(Xva))).numpy(), ova, val_p)
32         ts, tl = patient_scores(torch.sigmoid(net(torch.from_numpy(Xte))).numpy(), ote, test_p)
33
34     print(f"val  AUC {roc_auc_score(vl, vs):.3f}    test AUC {roc_auc_score(tl, ts):.3f}")
35     print(f"\ntest set: {int(tl.sum())} with muzmur, {int((1-tl).sum())} without\n")
36     print(f"{'target sens (val)':>18} {'thr':>7} | {'test sens':>9} {'test spec':>9} {'missed':>7} {'over-referred':>14}")
37     rows = []
38     for target in (0.85, 0.90, 0.925, 0.95, 0.975, 1.00):
39         order = np.sort(np.unique(vs))[:-1]; thr = order[-1]
40         for t in order:
41             p = vs >= t
42             tp = ((p == 1) & (vl == 1)).sum(); fn = ((p == 0) & (vl == 1)).sum()
43             if tp / max(tp + fn, 1e-9) >= target: thr = t; break
44             p = ts >= thr
45             tp = float(((p==1)&(tl==1)).sum()); tn = float(((p==0)&(tl==0)).sum())
46             fp = float(((p==1)&(tl==0)).sum()); fn = float(((p==0)&(tl==1)).sum())
47             sens = tp/max(tp+fn,1e-9); spec = tn/max(tn+fp,1e-9)
48             print(f"{'target':>18.3f} {'thr':>7.4f} | {'sens':>9.3f} {'spec':>9.3f} {'int(fn):>7} {'int(fp):>14}")
49             rows.append({"target": target, "threshold": float(thr), "test_sensitivity": sens,
50                         "test_specificity": spec, "missed": int(fn), "over_referred": int(fp)})
51     (HERE / "operating_points.json").write_text(json.dumps(rows, indent=1))
52
53 if __name__ == "__main__":
54     main()

```

```

1  """Export to ONNX and check the exported model agrees with PyTorch.
2
3  Inference runs in the browser, so the model that ships is this file's output,
4  not the checkpoint. If the two disagree, the thing users get is not the thing
5  that was measured -- so the agreement is asserted here rather than assumed.
6  """
7
8  from __future__ import annotations
9
10 import json
11 from pathlib import Path
12
13 import numpy as np
14 import onnxruntime as ort
15 import torch
16
17 import features as F
18 from model import MuzmurNet
19
20 HERE = Path(__file__).resolve().parent
21 OUT = HERE / "web" / "muzmur.onnx"
22
23
24 def main():
25     meta = json.loads((HERE / "metrics.json").read_text())
26     width = meta["width"]
27     # The shipped threshold is the operating point chosen from the sweep, not
28     # whatever train.py happened to land on. evaluate.py must have run first.
29     ops = json.loads((HERE / "operating_points.json").read_text())
30     chosen = next(o for o in ops if abs(o["target"] - 0.95) < 1e-9)
31     net = MuzmurNet(F.N_MELS)
32     net.load_state_dict(torch.load(HERE / "muzmur_net.pt", map_location="cpu"))
33     net.eval()
34
35     dummy = torch.randn(1, 1, F.N_MELS, width)
36     OUT.parent.mkdir(parents=True, exist_ok=True)
37     torch.onnx.export(
38         net, dummy, str(OUT),
39         input_names=["segment"], output_names=["logit"],
40         dynamic_axes={"segment": {0: "batch"}, "logit": {0: "batch"}},
41         opset_version=17,
42         dynamo=False, # keep the weights inside one file for the browser
43     )
44
45     sess = ort.InferenceSession(str(OUT), providers=["CPUExecutionProvider"])
46     probe = np.random.randn(5, 1, F.N_MELS, width).astype(np.float32)
47     with torch.no_grad():
48         want = net(torch.from_numpy(probe)).numpy()
49         got = sess.run(None, {"segment": probe})[0]
50         delta = float(np.max(np.abs(want - got)))
51         print(f"exported {OUT.name} ({OUT.stat().st_size/1024:.0f} KB)")
52         print(f"max |pytorch - onnx| on random input: {delta:.2e}")
53         if delta > 1e-4:
54             raise SystemExit("ONNX output disagrees with PyTorch; refusing to ship it")
55
56     (HERE / "web" / "model_meta.json").write_text(json.dumps({
57         "n_mels": F.N_MELS, "width": width, "sample_rate": F.SAMPLE_RATE,
58         "seg_seconds": F.SEG_SECONDS, "hop_seconds": F.HOP_SECONDS,
59         "n_fft": F.N_FFT, "hop": F.HOP, "fmin": F.FMIN, "fmax": F.FMAX,
60         "threshold": chosen["threshold"],
61         "chosen_operating_point": chosen, "metrics": {"val": meta["val"], "test": meta["test"]},
62     }, indent=1))
63     print("wrote web/model_meta.json")
64
65
66 if __name__ == "__main__":
67     main()

```

```
1  """Assert the browser features equal the Python features.
2
3  The model's held-out numbers were measured on the Python pipeline. If these two
4  diverge, those numbers describe a system nobody is running.
5  """
6  import json, sys
7  from pathlib import Path
8  import numpy as np
9  import features as F
10
11 wav, js_path = sys.argv[1], sys.argv[2]
12 js = json.loads(Path(js_path).read_text())
13 py = F.recording_features(wav) # (segs, 1, mels, frames)
14 jsarr = np.array(js["segments"], dtype=np.float32)
15 jsarr = jsarr.reshape(len(js["segments"]), js["nMels"], js["frames"])
16
17 print(f"python: {py.shape[0]} segments, {py.shape[2]}x{py.shape[3]}")
18 print(f"js      : {jsarr.shape[0]} segments, {jsarr.shape[1]}x{jsarr.shape[2]}")
19 assert py.shape[0] == jsarr.shape[0], "segment count differs"
20 w = min(py.shape[3], jsarr.shape[2])
21 a, b = py[:, 0, :, :w], jsarr[:, :, :w]
22 diff = np.abs(a - b)
23 corr = np.corrcoef(a.ravel(), b.ravel())[0, 1]
24 print(f"max |py - js| = {diff.max():.5f}   mean = {diff.mean():.6f}   corr = {corr:.6f}")
25 if diff.max() > 0.05:
26     raise SystemExit(f"FEATURES DIVERGE (max {diff.max():.4f}) - the browser would see different data")
27 print("features agree")
```

```

1  /* Compute log-mel for one recording with the browser code, dump as JSON. */
2  import fs from 'fs';
3  import * as dsp from './web/dsp.js';
4
5  const path = process.argv[2];
6  const buf = fs.readFileSync(path);
7  // minimal PCM16 WAV reader
8  let off = 12, fmt = null, dataOff = 0, dataLen = 0;
9  while (off < buf.length - 8) {
10   const id = buf.toString('ascii', off, off + 4);
11   const sz = buf.readUInt32LE(off + 4);
12   if (id === 'fmt ') fmt = { channels: buf.readUInt16LE(off + 10), rate: buf.readUInt32LE(off + 12), bits: buf.readUInt16LE(off +
22) };
13   if (id === 'data') { dataOff = off + 8; dataLen = sz; break; }
14   off += 8 + sz + (sz % 2);
15 }
16 const n = Math.floor(dataLen / 2 / fmt.channels);
17 let x = new Float32Array(n);
18 for (let i = 0; i < n; i++) {
19   let acc = 0;
20   for (let c = 0; c < fmt.channels; c++) acc += buf.readInt16LE(dataOff + (i * fmt.channels + c) * 2);
21   x[i] = acc / fmt.channels;
22 }
23 x = dsp.normaliseAudio(dsp.resample(x, fmt.rate, dsp.SR));
24 const starts = dsp.segmentIndices(x.length);
25 const segs = starts.map((s) => {
26   let seg = x.subarray(s, s + dsp.SEG_LEN);
27   if (seg.length < dsp.SEG_LEN) { const p = new Float32Array(dsp.SEG_LEN); p.set(seg); seg = p; }
28   return dsp.logMel(seg);
29 });
30 fs.writeFileSync(process.argv[3], JSON.stringify({
31   rate: fmt.rate, samples: n, starts, nMels: segs[0].nMels, frames: segs[0].frames,
32   segments: segs.map((s) => Array.from(s.data)),
33 }));
34 console.log(`js: ${segs.length} segments, ${segs[0].nMels}x${segs[0].frames}`);

```

```

1  """The check that actually matters: does the browser produce the same VERDICT?
2
3  Tiny per-cell differences in log-mel are expected, because log amplifies noise
4  in near-silent mel bins. What must not differ is the score the model returns,
5  because that is what a clinician acts on.
6  """
7  import json, subprocess, sys, tempfile
8  from pathlib import Path
9  import numpy as np, onnxruntime as ort
10 import features as F
11
12 sess = ort.InferenceSession("web/muxmur.onnx", providers=["CPUExecutionProvider"])
13 thr = json.loads(Path("metrics.json").read_text())["threshold"]
14 rows = []
15 for wav in sorted(Path("web/samples").glob("*.wav")):
16     out = Path(tempfile.mktemp(suffix=".json"))
17     subprocess.run(["node", "verify_dsp.mjs", str(wav), str(out)],
18                   check=True, capture_output=True)
19     js = json.loads(out.read_text())
20     jsf = np.array(js["segments"], np.float32).reshape(-1, 1, js["nMels"], js["frames"])
21     pyf = F.recording_features(str(wav))
22     w = min(jsf.shape[3], pyf.shape[3])
23     sp = 1 / (1 + np.exp(-sess.run(None, {"segment": pyf[:, :, :w]})[0])))
24     sj = 1 / (1 + np.exp(-sess.run(None, {"segment": jsf[:, :, :w]})[0])))
25     p, j = float(sp.max()), float(sj.max())
26     rows.append((wav.name, p, j, abs(p - j), (p >= thr) == (j >= thr)))
27
28 print(f'{"recording":30} {"python":>8} {"browser":>8} {"delta":>8} same verdict')
29 worst = 0.0
30 for name, p, j, d, same in rows:
31     worst = max(worst, d)
32     print(f'{"name":30} {"p":8.4f} {"j":8.4f} {"d":8.5f} {"yes" if same else "NO"}')
33 print(f"\nlargest score difference: {worst:.5f}")
34 if not all(r[4] for r in rows):
35     raise SystemExit("browser and python disagree on a verdict")
36 if worst > 0.02:
37     raise SystemExit(f"scores drift by {worst:.4f}; too much to ship")
38 print("browser and python agree on every sample")

```

```

1  /* Feature extraction, ported from features.py.
2  *
3  * This must match the Python exactly. The model was measured on features
4  * Python produced; if the browser computes something different, the held-out
5  * numbers describe a system nobody is running. verify_dsp.py checks the two
6  * against each other and the check is part of the build, not a courtesy.
7  */
8  export const SR = 4000;
9  export const SEG_SECONDS = 4.0;
10 export const SEG_LEN = SR * SEG_SECONDS;
11 export const HOP_SECONDS = 2.0;
12 export const N_MELS = 64;
13 export const N_FFT = 256;
14 export const HOP = 128;
15 export const FMIN = 25.0;
16 export const FMAX = 1000.0;
17 export const MAX_SEGMENTS = 8;
18
19 const hzToMel = (f) => 2595 * Math.log10(1 + f / 700);
20 const melToHz = (m) => 700 * (10 ** (m / 2595) - 1);
21
22 export function melFilterbank(sr = SR, nFft = N_FFT, nMels = N_MELS, fmin = FMIN, fmax = FMAX) {
23   const lo = hzToMel(fmin), hi = hzToMel(fmax);
24   const edges = Array.from({ length: nMels + 2 }, (_, i) => melToHz(lo + ((hi - lo) * i) / (nMels + 1)));
25   const bins = nFft / 2 + 1;
26   const freqs = Array.from({ length: bins }, (_, i) => (i * sr) / 2 / (bins - 1));
27   const fb = [];
28   for (let m = 0; m < nMels; m++) {
29     const [a, b, c] = [edges[m], edges[m + 1], edges[m + 2]];
30     const row = new Float32Array(bins);
31     for (let k = 0; k < bins; k++) {
32       const left = (freqs[k] - a) / Math.max(b - a, 1e-9);
33       const right = (c - freqs[k]) / Math.max(c - b, 1e-9);
34       row[k] = Math.max(Math.min(left, right), 0);
35     }
36     fb.push(row);
37   }
38   return fb;
39 }
40 const FB = melFilterbank();
41
42 /* scipy.signal.stft applies a Hann window scaled so the transform is unitary in
43 * amplitude. The scale factor matters: it moves every log-mel value by a
44 * constant, which per-segment normalisation would hide -- but only if the
45 * normalisation is applied, so it is reproduced rather than assumed away. */
46 function hann(n) {
47   const w = new Float32Array(n);
48   for (let i = 0; i < n; i++) w[i] = 0.5 - 0.5 * Math.cos((2 * Math.PI * i) / n);
49   return w;
50 }
51 const WIN = hann(N_FFT);
52 const WIN_SUM = WIN.reduce((a, b) => a + b, 0);
53
54 function dft(re, im) {
55   /* N_FFT is 256 and segments are few, so a direct radix-2 FFT is plenty. */
56   const n = re.length;
57   for (let i = 1, j = 0; i < n; i++) {
58     let bit = n >> 1;
59     for (; j & bit; bit >>= 1) j ^= bit;
60     j ^= bit;
61     if (i < j) { [re[i], re[j]] = [re[j], re[i]]; [im[i], im[j]] = [im[j], im[i]]; }
62   }
63   for (let len = 2; len <= n; len <= 1) {
64     const ang = (-2 * Math.PI) / len;
65     const wr = Math.cos(ang), wi = Math.sin(ang);
66     for (let i = 0; i < n; i += len) {
67       let cr = 1, ci = 0;
68       for (let k = 0; k < len / 2; k++) {
69         const ur = re[i + k], ui = im[i + k];
70         const vr = re[i + k + len / 2] * cr - im[i + k + len / 2] * ci;
71         const vi = re[i + k + len / 2] * ci + im[i + k + len / 2] * cr;
72         re[i + k] = ur + vr; im[i + k] = ui + vi;

```

```

73     re[i + k + len / 2] = ur - vr; im[i + k + len / 2] = ui - vi;
74     const ncr = cr * wr - ci * wi;
75     ci = cr * wi + ci * wr; cr = ncr;
76   }
77 }
78 }
79 }
80
81 export function logMel(seg) {
82   const bins = N_FFT / 2 + 1;
83   const frames = Math.floor((seg.length - N_FFT) / HOP) + 1;
84   const out = new Float32Array(N_MELS * frames);
85   const re = new Float64Array(N_FFT), im = new Float64Array(N_FFT);
86   const power = new Float64Array(bins);
87   for (let t = 0; t < frames; t++) {
88     const off = t * HOP;
89     for (let i = 0; i < N_FFT; i++) { re[i] = seg[off + i] * WIN[i]; im[i] = 0; }
90     dft(re, im);
91     for (let k = 0; k < bins; k++) {
92       const mag = Math.hypot(re[k], im[k]) / WIN_SUM;
93       /* scipy's one-sided spectrum doubles every bin except DC and Nyquist. */
94       const amp = (k === 0 || k === bins - 1) ? mag : mag * 2;
95       power[k] = amp * amp;
96     }
97     for (let m = 0; m < N_MELS; m++) {
98       const row = FB[m];
99       let acc = 0;
100       for (let k = 0; k < bins; k++) acc += row[k] * power[k];
101       out[m * frames + t] = Math.log(acc + 1e-8);
102     }
103   }
104   let mean = 0;
105   for (let i = 0; i < out.length; i++) mean += out[i];
106   mean /= out.length;
107   let v = 0;
108   for (let i = 0; i < out.length; i++) v += (out[i] - mean) ** 2;
109   const std = Math.sqrt(v / out.length);
110   for (let i = 0; i < out.length; i++) out[i] = (out[i] - mean) / (std + 1e-6);
111   return { data: out, nMels: N_MELS, frames };
112 }
113
114 export function normaliseAudio(x) {
115   let peak = 0;
116   for (let i = 0; i < x.length; i++) peak = Math.max(peak, Math.abs(x[i]));
117   if (peak === 0) return x;
118   const out = new Float32Array(x.length);
119   for (let i = 0; i < x.length; i++) out[i] = x[i] / peak;
120   return out;
121 }
122
123 export function resample(x, from, to) {
124   if (from === to) return x;
125   const n = Math.round((x.length * to) / from);
126   const out = new Float32Array(n);
127   for (let i = 0; i < n; i++) {
128     const pos = (i * (x.length - 1)) / (n - 1);
129     const a = Math.floor(pos), b = Math.min(a + 1, x.length - 1);
130     out[i] = x[a] + (x[b] - x[a]) * (pos - a);
131   }
132   return out;
133 }
134
135 export function segmentIndices(len) {
136   const hop = SR * HOP_SECONDS;
137   let starts = [];
138   for (let s = 0; s <= Math.max(0, len - SEG_LEN); s += hop) starts.push(s);
139   if (starts.length === 0) starts = [0];
140   if (starts.length > MAX_SEGMENTS) {
141     const pick = [];
142     for (let i = 0; i < MAX_SEGMENTS; i++) {
143       pick.push(starts[Math.round((i * (starts.length - 1)) / (MAX_SEGMENTS - 1))]);
144     }
145     starts = [...new Set(pick)];
146   }

```

```
147     return starts;
148 }
```

```

1  /* Muxmur -- browser screening aid.
2  *
3  * The feature pipeline here must stay identical to verify_dsp.mjs, which is the
4  * script that was checked cell by cell against the Python the model was measured
5  * on. Anything that drifts from it produces a number the held-out metrics do not
6  * describe, so this file imports dsp.js rather than reimplementing any of it.
7  */
8
9  import * as dsp from './dsp.js';
10 import * as ort from 'https://cdn.jsdelivr.net/npm/onnxruntime-web@1.20.1/dist/ort.wasm.min.mjs';
11
12 const ORT_DIST = 'https://cdn.jsdelivr.net/npm/onnxruntime-web@1.20.1/dist/';
13 ort.env.wasm.wasmPaths = ORT_DIST;
14 ort.env.wasm.numThreads = 1; /* no SharedArrayBuffer, so no cross-origin isolation needed */
15 ort.env.logLevel = 'error';
16
17 const MIC_MAX_SECONDS = 20;
18 const MIC_MIN_SECONDS = 4;
19
20 const LOCATIONS = {
21   AV: 'aortic valve',
22   MV: 'mitral valve',
23   PV: 'pulmonary valve',
24   TV: 'tricuspid valve',
25   Phc: 'other site',
26 };
27
28 const state = {
29   meta: null,
30   manifest: [],
31   session: null,
32   inputName: 'segment',
33   outputName: 'logit',
34   result: null,
35   selected: 0,
36   busy: false,
37   ctx: null,
38   source: null,
39   recording: null,
40 };
41
42 /* Exposed so a headless run can read what the page actually computed rather
43 * than scraping it back out of the rendered text. */
44 const probe = {
45   ready: false,
46   error: null,
47   result: null,
48   play: { playing: false, segment: null, plays: 0 },
49   consoleErrors: [],
50 };
51 window.__muxmur = probe;
52
53 const $ = (sel) => document.querySelector(sel);
54
55 function el(tag, cls, text) {
56   const n = document.createElement(tag);
57   if (cls) n.className = cls;
58   if (text !== undefined && text !== null) n.textContent = text;
59   return n;
60 }
61
62 const SVG_NS = 'http://www.w3.org/2000/svg';
63 const WAVE_PATH = 'M2 9h2.6l1.9-5 3 10 1.9-5H16';
64
65 /* Built rather than pasted as innerHTML: the icon is decoration, and nothing
66 * that renders a patient id should be going through an HTML parser. */
67 function icon(d) {
68   const svg = document.createElementNS(SVG_NS, 'svg');
69   svg.setAttribute('viewBox', '0 0 18 18');
70   svg.setAttribute('width', '16');
71   svg.setAttribute('height', '16');
72   svg.setAttribute('fill', 'none');

```

```

73   svg.setAttribute('stroke', 'currentColor');
74   svg.setAttribute('stroke-width', '1.5');
75   svg.setAttribute('stroke-linecap', 'round');
76   svg.setAttribute('stroke-linejoin', 'round');
77   const path = document.createElementNS(SVG_NS, 'path');
78   path.setAttribute('d', d);
79   svg.appendChild(path);
80   return svg;
81 }
82
83 const sigmoid = (z) => 1 / (1 + Math.exp(-z));
84 const fix = (v, n) => Number(v).toFixed(n);
85
86 /* Rounding a 0.9995 up to "1.000" would show a certainty the model never
87  * expressed, so scores that would round to either end gain a digit instead. */
88 function scoreText(v) {
89   const t = Number(v).toFixed(3);
90   if ((t === '1.000' && v < 1) || (t === '0.000' && v > 0)) return Number(v).toFixed(4);
91   return t;
92 }
93
94 function clock(seconds) {
95   const s = Math.max(0, Math.round(seconds));
96   return `${Math.floor(s / 60)}:${String(s % 60).padStart(2, '0')}`;
97 }
98
99 function tick() {
100   return new Promise((done) => requestAnimationFrame(() => setTimeout(done, 0)));
101 }
102
103 function setStatus(text) {
104   const box = $('#status');
105   if (!text) { box.hidden = true; box.textContent = ''; return; }
106   box.hidden = false;
107   box.textContent = text;
108 }
109
110 function setError(text) {
111   const box = $('#error');
112   if (!text) { box.hidden = true; box.textContent = ''; return; }
113   box.hidden = false;
114   box.textContent = text;
115   probe.error = text;
116 }
117
118 /* ---- audio decoding ----- */
119
120 /* Mirrors the WAV reader in verify_dsp.mjs. Kept rather than handing the bytes
121  * to decodeAudioData because the browser's decoder resamples to the context
122  * rate with a filter of its own, and the verified path resamples with
123  * dsp.resample. Same bytes in, same features out. */
124 function parseWav(buffer) {
125   const view = new DataView(buffer);
126   const bytes = new Uint8Array(buffer);
127   const tag = (off) => String.fromCharCode(bytes[off], bytes[off + 1], bytes[off + 2], bytes[off + 3]);
128   if (tag(0) !== 'RIFF' || tag(8) !== 'WAVE') return null;
129
130   let off = 12;
131   let fmt = null;
132   let dataOff = 0;
133   let dataLen = 0;
134   while (off < view.byteLength - 8) {
135     const id = tag(off);
136     const size = view.getUint32(off + 4, true);
137     if (id === 'fmt ') {
138       fmt = {
139         format: view.getUint16(off + 8, true),
140         channels: view.getUint16(off + 10, true),
141         rate: view.getUint32(off + 12, true),
142         bits: view.getUint16(off + 22, true),
143       };
144     }
145     if (id === 'data') { dataOff = off + 8; dataLen = Math.min(size, view.byteLength - dataOff); break; }
146     off += 8 + size + (size % 2);
147   }

```

```

148   if (!fmt || !dataOff) return null;
149
150   const ch = fmt.channels;
151   if (fmt.format === 1 && fmt.bits === 16) {
152     const n = Math.floor(dataLen / 2 / ch);
153     const out = new Float32Array(n);
154     for (let i = 0; i < n; i++) {
155       let acc = 0;
156       for (let c = 0; c < ch; c++) acc += view.getInt16(dataOff + (i * ch + c) * 2, true);
157       out[i] = acc / ch;
158     }
159     return { data: out, rate: fmt.rate };
160   }
161   if (fmt.format === 3 && fmt.bits === 32) {
162     const n = Math.floor(dataLen / 4 / ch);
163     const out = new Float32Array(n);
164     for (let i = 0; i < n; i++) {
165       let acc = 0;
166       for (let c = 0; c < ch; c++) acc += view.getFloat32(dataOff + (i * ch + c) * 4, true);
167       out[i] = acc / ch;
168     }
169     return { data: out, rate: fmt.rate };
170   }
171   return null;
172 }
173
174 async function decodeAny(buffer) {
175   const wav = parseWav(buffer);
176   if (wav) return wav;
177   const ctx = audioCtx();
178   const buf = await ctx.decodeAudioData(buffer.slice(0));
179   const n = buf.length;
180   const out = new Float32Array(n);
181   for (let c = 0; c < buf.numberOfChannels; c++) {
182     const ch = buf.getChannelData(c);
183     for (let i = 0; i < n; i++) out[i] += ch[i] / buf.numberOfChannels;
184   }
185   return { data: out, rate: buf.sampleRate };
186 }
187
188 function audioCtx() {
189   if (!state.ctx) state.ctx = new (window.AudioContext || window.webkitAudioContext)();
190   return state.ctx;
191 }
192
193 /* ---- the pipeline ----- */
194
195 async function analyse(mono, rate, source) {
196   if (state.busy) return;
197   state.busy = true;
198   setError('');
199   stopPlayback();
200   lockInputs(true);
201
202   try {
203     setStatus(`Resampling ${rate} Hz to ${dsp.SR} Hz...`);
204     await tick();
205     const audio = dsp.normaliseAudio(dsp.resample(mono, rate, dsp.SR));
206     const starts = dsp.segmentIndices(audio.length);
207
208     setStatus(`Computing log-mel features for ${starts.length} segment${starts.length === 1 ? '' : 's'}...`);
209     await tick();
210
211     const width = state.meta.width;
212     const nMels = dsp.N_MELS;
213     const feats = new Float32Array(starts.length * nMels * width);
214     for (let i = 0; i < starts.length; i++) {
215       let seg = audio.subarray(starts[i], starts[i] + dsp.SEG_LEN);
216       if (seg.length < dsp.SEG_LEN) {
217         const padded = new Float32Array(dsp.SEG_LEN);
218         padded.set(seg);
219         seg = padded;
220       }
221       const { data, frames } = dsp.logMel(seg);
222       const w = Math.min(frames, width);

```

```

223     const base = i * nMels * width;
224     for (let m = 0; m < nMels; m++) {
225         for (let t = 0; t < w; t++) feats[base + m * width + t] = data[m * frames + t];
226     }
227 }
228
229 setStatus('Running the model...');
230 await tick();
231
232 const tensor = new ort.Tensor('float32', feats, [starts.length, 1, nMels, width]);
233 const out = await state.session.run({ [state.inputName]: tensor });
234 const logits = out[state.outputName].data;
235 const scores = Array.from(logits, sigmoid);
236
237 let top = 0;
238 for (let i = 1; i < scores.length; i++) if (scores[i] > scores[top]) top = i;
239
240 state.result = {
241     source,
242     audio,
243     starts,
244     scores,
245     top,
246     score: scores[top],
247     seconds: audio.length / dsp.SR,
248     refer: scores[top] >= state.meta.threshold,
249 };
250 state.selected = top;
251
252 probe.result = {
253     source: source.label,
254     sample: source.sample ? source.sample.file : null,
255     score: state.result.score,
256     scores: scores.slice(),
257     top,
258     starts: starts.slice(),
259     threshold: state.meta.threshold,
260     refer: state.result.refer,
261 };
262
263 setStatus('');
264 renderResult();
265 $('#result').scrollIntoView({ block: 'nearest', behavior: 'smooth' });
266 } catch (err) {
267     setStatus('');
268     setError('Could not analyse this recording: ${err} && err.message ? err.message : err');
269 } finally {
270     state.busy = false;
271     lockInputs(false);
272 }
273 }
274
275 /* ---- rendering the result ----- */
276
277 function renderResult() {
278     const r = state.result;
279     const thr = state.meta.threshold;
280     const box = $('#result');
281     box.hidden = false;
282     box.textContent = '';
283
284     /* verdict */
285     const head = el('div', 'result-head');
286     head.appendChild(el('p', 'result-src', r.source.label));
287     const v = el('h3', `verdict${r.refer ? ' is-refer' : ''}`, r.refer ? 'Refer for echocardiography' : 'No referral indicated');
288     v.id = 'verdict';
289     head.appendChild(v);
290     head.appendChild(el('p', 'verdict-sub', r.refer
291         ? 'A murmur is audible enough in this recording to be worth imaging. This is a prompt to arrange an echocardiogram, not a
292         finding about the heart.'
293         : 'No murmur was audible enough in this recording to prompt imaging. This does not rule anything out; a clinician who suspects
294         disease should refer regardless.'));
295     box.appendChild(head);
296
297     box.appendChild(scaleBlock(r, thr));

```

```

296 box.appendChild(segmentBlock(r, thr));
297 if (r.source.sample) box.appendChild(truthBlock(r.source.sample, r.refer));
298 }
299
300 function scaleBlock(r, thr) {
301   const block = el('section', 'block');
302   block.appendChild(el('h4', 'block-h', 'Model score against the decision threshold'));
303
304   const scale = el('div', 'scale');
305
306   const above = el('div', 'scale-top');
307   const tagScore = el('div', 'scale-score');
308   tagScore.appendChild(el('b', null, scoreText(r.score)));
309   tagScore.appendChild(el('span', null, 'this recording'));
310   place(tagScore, r.score);
311   above.appendChild(tagScore);
312   scale.appendChild(above);
313
314   const mark = el('div', 'scale-mark');
315   const tri = el('span', 'scale-tri');
316   tri.style.left = `${r.score * 100}%`;
317   mark.appendChild(tri);
318   scale.appendChild(mark);
319
320   const track = el('div', 'scale-track');
321   const over = el('span', 'scale-over');
322   over.style.left = `${thr * 100}%`;
323   track.appendChild(over);
324
325   const thrLine = el('span', 'scale-thr');
326   thrLine.style.left = `${thr * 100}%`;
327   track.appendChild(thrLine);
328
329   const pin = el('span', 'scale-pin');
330   pin.style.left = `${r.score * 100}%`;
331   track.appendChild(pin);
332   scale.appendChild(track);
333
334   const ends = el('div', 'scale-ends');
335   ends.appendChild(el('span', null, '0.000'));
336   ends.appendChild(el('span', null, '1.000'));
337   scale.appendChild(ends);
338
339   const labels = el('div', 'scale-labels');
340   const tagThr = el('div', 'scale-tag tag-thr');
341   tagThr.appendChild(el('b', null, `threshold ${fix(thr, 3)}`));
342   tagThr.appendChild(el('span', null, r.refer ? 'refer above this' : 'no referral below this'));
343   place(tagThr, thr);
344   labels.appendChild(tagThr);
345   scale.appendChild(labels);
346
347   block.appendChild(scale);
348
349   const gap = Math.abs(r.score - thr);
350   const note = el('p', 'scale-note');
351   note.appendChild(document.createTextNode('This recording sits '));
352   note.appendChild(el('span', 'scale-gap', fix(gap, 3)));
353   note.appendChild(document.createTextNode(r.score >= thr
354     ? ' above the threshold. '
355     : ' below the threshold. '));
356   note.appendChild(document.createTextNode(gap < 0.1
357     ? 'That is a close call: a slightly different microphone, room, or moment in the recording could have moved it across. '
358     : ''));
359   note.appendChild(document.createTextNode('The score is the model\'s output for this recording. It is not a probability that this
child has rheumatic heart disease, and it should not be read as one.));
360   block.appendChild(note);
361
362   return block;
363 }
364
365 /* Labels near either end would otherwise run off the track, so they flip to the
366 * inside once they get close to an edge. */
367 function place(node, pos) {
368   const pct = pos * 100;
369   node.style.left = `${pct}%`;

```

```

370   if (pct > 60) { node.style.transform = 'translateX(-100%)'; node.style.textAlign = 'right'; }
371   else if (pct < 14) node.style.transform = 'none';
372   else node.style.transform = 'translateX(-50%)';
373 }
374
375 function segmentBlock(r, thr) {
376   const block = el('section', 'block');
377   block.appendChild(el('h4', 'block-h', 'The evidence'));
378
379   const lede = el('p', 'note', `This recording is ${fix(r.seconds, 1)} seconds long. It was cut into ${r.starts.length} four-second
segment${r.starts.length === 1 ? '' : 's'} spanning the whole of it, and the model scored each one on its own. The score above is the
highest of them, so that one segment is the evidence the decision rests on. It is here to listen to.`);
380   block.appendChild(lede);
381
382   const chart = el('div', 'segs');
383   const line = el('div', 'seg-thr');
384   line.style.bottom = `${thr * 100}%`;
385   line.appendChild(el('span', 'seg-thr-tag', 'threshold'));
386   chart.appendChild(line);
387
388   const axis = el('div', 'seg-axis');
389
390   r.scores.forEach((s, i) => {
391     const b = el('button', `seg${i} === r.top ? ' is-top' : ''`);
392     b.type = 'button';
393     b.setAttribute('aria-pressed', String(i === state.selected));
394     const startSec = r.starts[i] / dsp.SR;
395     b.title = `Segment ${i + 1}: ${clock(startSec)} to ${clock(startSec + dsp.SEG_SECONDS)}, score ${scoreText(s)}`;
396     b.setAttribute('aria-label', b.title);
397     const bar = el('span', 'seg-bar');
398     bar.style.height = `${Math.max(2, s * 100)}%`;
399     b.appendChild(bar);
400     b.addEventListener('click', () => selectSegment(i));
401     chart.appendChild(b);
402     axis.appendChild(el('span', null, clock(startSec)));
403   });
404
405   block.appendChild(chart);
406   block.appendChild(axis);
407
408   const pick = el('div', 'seg-pick');
409   const facts = el('div', 'seg-facts');
410   facts.id = 'seg-facts';
411   pick.appendChild(facts);
412   const play = el('button', 'btn btn-primary');
413   play.type = 'button';
414   play.id = 'play-btn';
415   play.addEventListener('click', togglePlay);
416   pick.appendChild(play);
417   block.appendChild(pick);
418
419   const sel = axis.children[state.selected];
420   if (sel) sel.classList.add('is-sel');
421
422   window.setTimeout(() => describeSelection(), 0);
423   return block;
424 }
425
426 function selectSegment(i) {
427   state.selected = i;
428   document.querySelectorAll('.seg').forEach((n, k) => n.setAttribute('aria-pressed', String(k === i)));
429   document.querySelectorAll('.seg-axis span').forEach((n, k) => n.classList.toggle('is-sel', k === i));
430   stopPlayback();
431   describeSelection();
432 }
433
434 function describeSelection() {
435   const r = state.result;
436   if (!r) return;
437   const i = state.selected;
438   const facts = $('#seg-facts');
439   const play = $('#play-btn');
440   if (!facts || !play) return;
441   const startSec = r.starts[i] / dsp.SR;
442   facts.textContent = '';

```

```

443 facts.appendChild(el('b', null, `Segment ${i + 1} of ${r.scores.length}  ${clock(startSec)} to ${clock(startSec +
dsp.SEG_SECONDS)}  score ${scoreText(r.scores[i])}`));
444 facts.appendChild(document.createTextNode(i === r.top
445   ? 'Highest-scoring segment. This is what set the result.'
446   : 'Selected segment. The result was set by segment ' + (r.top + 1) + '.'));
447 play.textContent = state.source ? 'Stop' : 'Play this segment';
448 play.classList.toggle('btn-rec', Boolean(state.source));
449 }
450
451 function truthBlock(sample, refer) {
452   const block = el('section', 'block');
453   block.appendChild(el('h4', 'block-h', 'Reference annotation'));
454
455   const row = el('dl', 'truth-row');
456   const cell = (term, value) => {
457     const d = el('div', 'truth-cell');
458     d.appendChild(el('dt', null, term));
459     d.appendChild(el('dd', null, value));
460     row.appendChild(d);
461   };
462   cell('Murmur', sample.truth);
463   cell('Outcome', sample.outcome);
464   cell('Age group', sample.age);
465   cell('Site', LOCATIONS[sample.location] || sample.location);
466   cell('Test set', sample.held_out ? 'held out' : 'not held out');
467   block.appendChild(row);
468
469   const call = el('p', 'truth-call');
470   let kind = 'call-unknown';
471   let verdict = '';
472   let rest = '';
473   if (sample.truth === 'Unknown') {
474     verdict = 'The annotators could not tell either.';
475     rest = ' Human experts listened to this recording and marked the murmur label Unknown. Recordings like it were excluded from
training, and the model has no way to return that answer, so it returns a number regardless. This one was not part of the held-out test
set.';
476   } else if (sample.truth === 'Present' && refer) {
477     kind = 'call-hit';
478     verdict = 'Correct referral.';
479     rest = ' A murmur was annotated as present, and the model scored this recording above the threshold. This child would have been
sent for an echocardiogram.';
480   } else if (sample.truth === 'Present' && !refer) {
481     kind = 'call-miss';
482     verdict = 'A miss.';
483     rest = ' A murmur was annotated as present, and the model scored this recording below the threshold. This child would have been
sent home. Misses are the cost that matters most here, and the threshold is set where it is to keep them rare rather than to make the tool
look accurate.';
484   } else if (sample.truth === 'Absent' && refer) {
485     kind = 'call-over';
486     verdict = 'An over-referral.';
487     rest = ' No murmur was annotated, and the model scored this recording above the threshold. This child would have been sent for
an echocardiogram that was not needed, which costs a scarce appointment and frightens a family.';
488   } else {
489     kind = 'call-hit';
490     verdict = 'Correct.';
491     rest = ' No murmur was annotated, and the model scored this recording below the threshold.';
492   }
493   call.classList.add(kind);
494   call.appendChild(el('b', null, verdict));
495   call.appendChild(document.createTextNode(rest));
496   block.appendChild(call);
497
498   if (probe.result) probe.result.call = kind;
499   return block;
500 }
501
502 /* ---- playback ----- */
503
504 function togglePlay() {
505   if (state.source) { stopPlayback(); return; }
506   playSegment(state.selected);
507 }
508
509 function playSegment(i) {
510   const r = state.result;

```

```

511   if (!r) return;
512   const ctx = audioCtx();
513   if (ctx.state === 'suspended') ctx.resume();
514
515   const start = r.starts[i];
516   const buf = ctx.createBuffer(1, dsp.SEG_LEN, dsp.SR);
517   const chunk = new Float32Array(dsp.SEG_LEN);
518   chunk.set(r.audio.subarray(start, Math.min(start + dsp.SEG_LEN, r.audio.length)));
519   buf.copyToChannel(chunk, 0);
520
521   const src = ctx.createBufferSource();
522   src.buffer = buf;
523   src.connect(ctx.destination);
524   src.onended = () => { if (state.source === src) stopPlayback(); };
525   src.start();
526
527   state.source = src;
528   probe.play = { playing: true, segment: i, plays: probe.play.plays + 1 };
529   describeSelection();
530 }
531
532 function stopPlayback() {
533   if (state.source) {
534     try { state.source.onended = null; state.source.stop(); } catch (err) { /* already ended */ }
535     state.source = null;
536   }
537   probe.play.playing = false;
538   describeSelection();
539 }
540
541 /* ---- sources ----- */
542
543 function lockInputs(on) {
544   document.querySelectorAll('.sample').forEach((b) => { b.disabled = on; });
545   const mic = $('#mic-btn');
546   if (!state.recording) mic.disabled = on;
547 }
548
549 async function runSample(i) {
550   const sample = state.manifest[i];
551   document.querySelectorAll('.sample').forEach((b, k) => b.setAttribute('aria-pressed', String(k === i)));
552   try {
553     setStatus('Loading recording...');
554     const res = await fetch(`samples/${sample.file}`);
555     if (!res.ok) throw new Error(`sample ${sample.file} returned ${res.status}`);
556     const { data, rate } = await decodeAny(await res.arrayBuffer());
557     await analyse(data, rate, {
558       label: 'Sample recording   patient ${patientId(sample.file)}',
559       sample,
560     });
561   } catch (err) {
562     setStatus('');
563     setError(`Could not load that recording: ${err} && err.message ? err.message : err`);
564   }
565 }
566
567 function patientId(file) {
568   const m = file.match(/(\d+)\.wav$/);
569   return m ? m[1] : file;
570 }
571
572 /* The card shows the patient id, age and site and nothing else. The manifest
573 * also carries the kind -- caught, missed, false-alarm -- and putting that on
574 * the card would tell the reader the answer before the model has said a word. */
575 function renderSamples() {
576   const list = $('#samples');
577   list.textContent = '';
578   state.manifest.forEach((s, i) => {
579     const b = el('button', 'card sample');
580     b.type = 'button';
581     b.disabled = true;
582     b.setAttribute('aria-pressed', 'false');
583     b.dataset.index = String(i);
584
585     const head = el('div', 'card-head');

```

```

586     const tile = el('span', 'tile');
587     tile.setAttribute('aria-hidden', 'true');
588     tile.appendChild(icon(WAVE_PATH));
589     head.appendChild(tile);
590
591     const text = el('div');
592     text.appendChild(el('div', 'card-t', `Patient ${patientId(s.file)}`));
593     text.appendChild(el('div', 'card-s', `${s.age}, ${LOCATIONS[s.location] || s.location}`));
594     head.appendChild(text);
595
596     b.appendChild(head);
597     b.addEventListener('click', () => runSample(i));
598     list.appendChild(b);
599   });
600 }
601
602 /* ---- microphone ----- */
603
604 const WORKLET = `
605 class Tap extends AudioWorkletProcessor {
606   process(inputs) {
607     const ch = inputs[0] && inputs[0][0];
608     if (ch && ch.length) this.port.postMessage(new Float32Array(ch));
609     return true;
610   }
611 }
612 registerProcessor('murmur-tap', Tap);
613 `;
614
615 async function startRecording() {
616   setError('');
617   let stream;
618   try {
619     /* Echo cancellation, noise suppression and auto gain are all tuned for
620      * speech and would eat a heart sound alive, so they are turned off. */
621     stream = await navigator.mediaDevices.getUserMedia({
622       audio: {
623         echoCancellation: false,
624         noiseSuppression: false,
625         autoGainControl: false,
626         channelCount: 1,
627       },
628     });
629   } catch (err) {
630     setError(`No microphone available: ${err && err.message ? err.message : err}. The sample recordings work without one.`);
631     return;
632   }
633
634   const ctx = audioCtx();
635   await ctx.resume();
636   const src = ctx.createMediaStreamSource(stream);
637   const sink = ctx.createGain();
638   sink.gain.value = 0;
639   sink.connect(ctx.destination);
640
641   const chunks = [];
642   const onChunk = (block) => {
643     chunks.push(block);
644     let peak = 0;
645     for (let i = 0; i < block.length; i++) peak = Math.max(peak, Math.abs(block[i]));
646     const fill = `#mic-level`;
647     if (fill) fill.style.width = `${Math.min(100, peak * 140)}%`;
648   };
649
650   let node = null;
651   if (ctx.audioWorklet) {
652     try {
653       const url = URL.createObjectURL(new Blob([WORKLET], { type: 'text/javascript' }));
654       await ctx.audioWorklet.addModule(url);
655       URL.revokeObjectURL(url);
656       node = new AudioWorkletNode(ctx, 'murmur-tap');
657       node.port.onmessage = (e) => onChunk(e.data);
658     } catch (err) {
659       node = null;
660     }

```

```

661 }
662 if (!node) {
663     node = ctx.createScriptProcessor(4096, 1, 1);
664     node.onaudioprocess = (e) => onChunk(new Float32Array(e.inputBuffer.getChannelData(0)));
665 }
666 src.connect(node);
667 node.connect(sink);
668
669 const started = performance.now();
670 const timer = window.setInterval(() => {
671     const secs = (performance.now() - started) / 1000;
672     $('#mic-state').textContent = `Recording ${clock(secs)} (stops at ${clock(MIC_MAX_SECONDS)})`;
673     if (secs >= MIC_MAX_SECONDS) stopRecording();
674 }, 100);
675
676 state.recording = { stream, src, node, sink, chunks, timer, started, rate: ctx.sampleRate };
677
678 const btn = $('#mic-btn');
679 btn.textContent = 'Stop and analyse';
680 btn.classList.add('btn-rec');
681 $('#mic-meter').hidden = false;
682 $('#mic-state').textContent = 'Recording 0:00';
683 document.querySelectorAll('.sample').forEach((b) => { b.disabled = true; });
684 }
685
686 async function stopRecording() {
687     const rec = state.recording;
688     if (!rec) return;
689     state.recording = null;
690     window.clearInterval(rec.timer);
691
692     try { rec.src.disconnect(); rec.node.disconnect(); rec.sink.disconnect(); } catch (err) { /* already down */ }
693     if (rec.node.port) rec.node.port.onmessage = null;
694     rec.node.onaudioprocess = null;
695     rec.stream.getTracks().forEach((t) => t.stop());
696
697     const btn = $('#mic-btn');
698     btn.textContent = 'Record from microphone';
699     btn.classList.remove('btn-rec');
700     $('#mic-meter').hidden = true;
701     $('#mic-level').style.width = '0';
702     document.querySelectorAll('.sample').forEach((b) => { b.disabled = false; });
703
704     let total = 0;
705     rec.chunks.forEach((c) => { total += c.length; });
706     const seconds = total / rec.rate;
707     if (seconds < MIC_MIN_SECONDS) {
708         $('#mic-state').textContent = '';
709         setError(`That recording is only ${fix(seconds, 1)} seconds. The model needs at least ${MIC_MIN_SECONDS} seconds of audio.`);
710         return;
711     }
712
713     const mono = new Float32Array(total);
714     let at = 0;
715     rec.chunks.forEach((c) => { mono.set(c, at); at += c.length; });
716
717     $('#mic-state').textContent = `Captured ${fix(seconds, 1)} s at ${rec.rate} Hz`;
718     document.querySelectorAll('.sample').forEach((b) => b.setAttribute('aria-pressed', 'false'));
719     await analyse(mono, rec.rate, { label: `Microphone ${fix(seconds, 1)} seconds`, sample: null });
720 }
721
722 /* ---- how it works ----- */
723
724 /* metrics.test is reported at whatever threshold training settled on. The
725 * threshold this tool ships with is the chosen operating point, which is a
726 * different place on the same curve -- so the panel quotes that point's
727 * sensitivity and specificity, not the ones next to them in the file. Quoting
728 * the wrong pair would describe a tool nobody is running. */
729 function heldOut(m) {
730     const t = m.metrics.test;
731     const muymurs = t.tp + t.fn;
732     const clean = t.tn + t.fp;
733     const op = m.chosen_operating_point;
734     if (!op) return { n: t.n, auc: t.auc, muymurs, clean, tp: t.tp, fn: t.fn, tn: t.tn, fp: t.fp,
735         sensitivity: t.sensitivity, specificity: t.specificity };

```

```

736   return {
737     n: t.n, auc: t.auc, murmurs, clean,
738     fn: op.missed, fp: op.over_referred,
739     tp: murmurs - op.missed, tn: clean - op.over_referred,
740     sensitivity: op.test_sensitivity, specificity: op.test_specificity,
741   };
742 }
743
744 function renderAbout() {
745   const m = state.meta;
746   const t = heldOut(m);
747
748   const stages = [
749     ['Resample and normalise',
750       'The recording is mixed to mono, resampled to a low rate, and divided by its own peak so that a quiet recording and a loud one
arrive at the model on the same footing.',
751       `${m.sample_rate} Hz, peak normalised`],
752     ['Log-mel spectrogram',
753       'Each segment becomes a picture of how sound energy is spread across frequency over time, on a mel scale bounded to the band
where heart sounds and murmurs live. Every picture is standardised to zero mean and unit variance.',
754       `${m.n_mels} mel bands, ${m.fmin}-${m.fmax} Hz, ${m.n_fft}-point FFT, hop ${m.hop}`],
755     ['A small CNN scores each segment',
756       'A four-block convolutional network reads one spectrogram at a time and returns a single number for that four seconds of
audio. It sees no more of the recording than that. Long recordings are sampled down to eight segments spread across their length rather than
scored window by window.',
757       `${m.seg_seconds} s segments every ${m.hop_seconds} s, at most 8 per recording, ${m.n_mels} by ${m.width} input`],
758     ['Take the maximum across segments',
759       'A murmur need only be audible somewhere, so the recording takes the highest segment score rather than the average. Averaging
would let a few clean seconds bury a murmur that was plainly there.',
760       `max over segments, refer at score >= ${fix(m.threshold, 4)}`],
761   ];
762
763   const ol = $('#stages');
764   ol.textContent = '';
765   stages.forEach((s, i) => {
766     const li = el('li', 'stage');
767     li.appendChild(el('div', 'stage-n', String(i + 1).padStart(2, '0')));
768     const body = el('div');
769     body.appendChild(el('div', 'stage-t', s[0]));
770     body.appendChild(el('div', 'stage-d', s[1]));
771     body.appendChild(el('div', 'stage-k', s[2]));
772     li.appendChild(body);
773     ol.appendChild(li);
774   });
775
776   const figures = [
777     ['Held-out patients', String(t.n), 'never seen in training'],
778     ['AUC', fix(t.auc, 3), 'ranking, before any threshold'],
779     ['Sensitivity', fix(t.sensitivity, 3), `${t.tp} of ${t.murmurs} murmurs caught'],
780     ['Specificity', fix(t.specificity, 3), `${t.tn} of ${t.clean} cleared'],
781   ];
782   const dl = $('#figures');
783   dl.textContent = '';
784   figures.forEach(([term, value, sub]) => {
785     const d = el('div');
786     d.appendChild(el('dt', null, term));
787     const dd = el('dd', null, value);
788     dd.appendChild(el('span', 'fig-sub', sub));
789     d.appendChild(dd);
790     dl.appendChild(d);
791   });
792
793   const miss = t.fn === 1 ? '1 murmur' : `${t.fn} murmurs`;
794   const over = t.fp === 1 ? '1 child' : `${t.fp} children`;
795   $('#limits-prose').textContent =
796     `On ${t.n} held-out patients the model reached an AUC of ${fix(t.auc, 3)}. At the threshold this tool uses, ${fix(m.threshold,
4)}, it caught ${t.tp} of the ${t.murmurs} patients annotated with a murmur and cleared ${t.tn} of the ${t.clean} without one. That means it
missed ${miss}, and would have sent ${over} for an echocardiogram they did not need. Both of those are the price of this operating point,
and neither is a rounding error.`;
797 }
798
799 /* ---- views ----- */
800
801 const VIEWS = ['screen', 'evidence', 'about', 'limits'];
802

```

```

803 function showView(name) {
804   document.querySelectorAll('.nav-item').forEach((t) => {
805     const on = t.dataset.view === name;
806     t.classList.toggle('is-on', on);
807     if (on) t.setAttribute('aria-current', 'page');
808     else t.removeAttribute('aria-current');
809   });
810   VIEWS.forEach((v) => {
811     const node = $('#view-${v}');
812     if (node) node.hidden = v !== name;
813   });
814   window.scrollTo({ top: 0 });
815 }
816
817 /* ---- boot ----- */
818
819 async function boot() {
820   /* Both the rail nav and the primary button carry data-view. */
821   document.querySelectorAll('[data-view]').forEach((t) => {
822     t.addEventListener('click', () => showView(t.dataset.view));
823   });
824   $('#mic-btn').addEventListener('click', () => {
825     if (state.recording) stopRecording();
826     else startRecording();
827   });
828
829   try {
830     const [meta, manifest] = await Promise.all([
831       fetch('model_meta.json').then((r) => r.json()),
832       fetch('samples/manifest.json').then((r) => r.json()),
833     ]);
834     state.meta = meta;
835     state.manifest = manifest;
836     probe.threshold = meta.threshold;
837
838     renderSamples();
839     renderAbout();
840
841     $('#boot').textContent = 'Loading model (about 600 KB, once)...';
842     state.session = await ort.InferenceSession.create('murmur.onnx', {
843       executionProviders: ['wasm'],
844       graphOptimizationLevel: 'all',
845     });
846     state.inputName = state.session.inputNames[0];
847     state.outputName = state.session.outputNames[0];
848
849     $('#boot').hidden = true;
850     document.querySelectorAll('.sample').forEach((b) => { b.disabled = false; });
851     probe.ready = true;
852   } catch (err) {
853     const msg = err && err.message ? err.message : String(err);
854     $('#boot').textContent = `The model could not be loaded: ${msg}`;
855     probe.error = msg;
856   }
857 }
858
859 window.addEventListener('error', (e) => probe.consoleErrors.push(String(e.message)));
860 window.addEventListener('unhandledrejection', (e) => probe.consoleErrors.push(String(e.reason)));
861
862 boot();

```

```

1  <!doctype html>
2  <html lang="en-GB">
3  <head>
4  <meta charset="utf-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1">
6  <title>Murmur -- heart-sound screening aid</title>
7  <meta name="description" content="A screening aid that listens to a heart-sound recording and says whether a murmur is likely enough
to warrant echocardiography. It does not diagnose.">
8  <link rel="preconnect" href="https://fonts.googleapis.com">
9  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10 <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Mono:wght@400;500&family=Inter:wght@400;500;600&display=swap"
rel="stylesheet">
11 <link rel="stylesheet" href="app.css">
12 </head>
13 <body>
14 <a class="skip" href="#main">Skip to content</a>
15
16 <div class="app">
17
18   <aside class="rail">
19     <div class="rail-top">
20       <div class="brand">
21         <span class="brand-tile" aria-hidden="true">
22           <svg viewBox="0 0 18 18" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.6" stroke-
linecap="round" stroke-linejoin="round"><path d="M2 9h2.6l1.9-5 3 10 1.9-5H16"/></svg>
23         </span>
24         <span class="brand-name">Murmur</span>
25       </div>
26
27       <button type="button" class="btn btn-primary rail-cta" data-view="screen">Screen a recording</button>
28
29       <nav class="nav" aria-label="Sections">
30         <button type="button" class="nav-item is-on" data-view="screen" aria-current="page">
31           <svg class="nav-ico" viewBox="0 0 18 18" width="17" height="17" fill="none" stroke="currentColor" stroke-width="1.5"
stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"><path d="M2 9h2.6l1.9-5 3 10 1.9-5H16"/></svg>
32           <span>Screen</span>
33         </button>
34         <button type="button" class="nav-item" data-view="evidence">
35           <svg class="nav-ico" viewBox="0 0 18 18" width="17" height="17" fill="none" stroke="currentColor" stroke-width="1.5"
stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"><path d="M3 15.2v9.4M9 15.2v3.2M15 15.2v-4.4"/></svg>
36           <span>Evidence</span>
37         </button>
38         <button type="button" class="nav-item" data-view="about">
39           <svg class="nav-ico" viewBox="0 0 18 18" width="17" height="17" fill="none" stroke="currentColor" stroke-width="1.5"
stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"><path d="M9 2.2 2.2 6 9 9.8 15.8 6 9 2.2"/><path d="m2.2 11.6 6.8 3.8
6.8-3.8"/></svg>
40           <span>How it works</span>
41         </button>
42         <button type="button" class="nav-item" data-view="limits">
43           <svg class="nav-ico" viewBox="0 0 18 18" width="17" height="17" fill="none" stroke="currentColor" stroke-width="1.5"
stroke-linecap="round" stroke-linejoin="round" aria-hidden="true"><circle cx="9" cy="9" r="6.6"/><path d="M9 5.4v4.1"/><path d="M9
12.3h.01"/></svg>
44           <span>Limits</span>
45         </button>
46       </nav>
47     </div>
48
49   <div class="rail-foot">
50     <div class="usage">
51       <div class="usage-row"><span>Model</span><span class="num">575 KB</span></div>
52       <div class="usage-bar" aria-hidden="true"><span style="width:56%"></span></div>
53     </div>
54     <div class="usage">
55       <div class="usage-row"><span>Test AUC</span><span class="num">0.957</span></div>
56       <div class="usage-bar" aria-hidden="true"><span style="width:96%"></span></div>
57     </div>
58     <p class="rail-note">Measured once on 174 held-out patients. Nothing leaves this device.</p>
59   </div>
60 </aside>
61
62 <div class="main">
63

```

```

64 <header class="topbar">
65   <div class="topbar-field" aria-hidden="true">
66     <svg viewBox="0 0 18 18" width="15" height="15" fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round">
<circle cx="8" cy="8" r="5.1"/><path d="m11.8 11.8 3 3"/></svg>
67     <span>CirCor DigiScope - 942 patients, 5 bundled recordings</span>
68   </div>
69   <div class="topbar-acts">
70     <span class="topbar-act">Runs in your browser</span>
71     <span class="topbar-act">Not a medical device</span>
72   </div>
73 </header>
74
75 <main id="main" class="content">
76
77   <section id="view-screen" class="view">
78     <div class="page-head">
79       <h1 class="page-title">Does this heart sound need an echo?</h1>
80       <p class="page-lede">This tool does not diagnose. It listens to a heart-sound recording
81         and says whether a murmur is audible enough to warrant an echocardiogram. A murmur is
82         not rheumatic heart disease, and the decision to refer belongs to a clinician.</p>
83     </div>
84
85     <div id="boot" class="boot" role="status">Loading model...</div>
86
87     <section class="sec" aria-labelledby="h-samples">
88       <h2 class="sec-h" id="h-samples">Pick a recording</h2>
89       <p class="sec-note">Five recordings from the CirCor DigiScope dataset. The reference
90         annotation for each is hidden until after the model has run.</p>
91       <div id="samples" class="cardrow"></div>
92     </section>
93
94     <section class="sec" aria-labelledby="h-mic">
95       <h2 class="sec-h" id="h-mic">Or record from a microphone</h2>
96       <div class="card card-wide">
97         <div class="card-head">
98           <span class="tile" aria-hidden="true">
99             <svg viewBox="0 0 18 18" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.5" stroke-
linecap="round" stroke-linejoin="round"><rect x="6.4" y="2" width="5.2" height="8.2" rx="2.6"/><path d="M3.8 8.6a5.2 5.2 0 0 10.4 0"/>
<path d="M9 13.8V16"/></svg>
100           </span>
101         </div>
102         <div class="card-t">Live capture</div>
103         <div class="card-s">Eight seconds or more</div>
104       </div>
105     </div>
106     <p class="card-body">Hold the microphone firmly against the chest, over the upper left
107       side of the sternum, in as quiet a room as you can find. Stay still.</p>
108     <div class="mic">
109       <button type="button" id="mic-btn" class="btn btn-primary">Record from microphone</button>
110       <div id="mic-state" class="mic-state" role="status" aria-live="polite"></div>
111       <div id="mic-meter" class="meter" hidden><span id="mic-level" class="meter-fill"></span></div>
112     </div>
113     <p class="card-warn">The held-out numbers on this model come from clinical recordings
114       made with a digital stethoscope. Performance on a phone microphone is unmeasured, so
115       treat a microphone result as a demonstration, not a screen.</p>
116   </div>
117 </section>
118
119   <div id="status" class="status" role="status" aria-live="polite" hidden></div>
120   <div id="error" class="error" role="alert" hidden></div>
121
122   <section id="result" class="result" hidden aria-labelledby="verdict"></section>
123 </section>
124
125   <section id="view-evidence" class="view" hidden>
126     <div class="page-head">
127       <h1 class="page-title">What the numbers are</h1>
128       <p class="page-lede">Measured once, on patients the model never saw during training or
129         threshold selection. Nothing here was tuned after it was read.</p>
130     </div>
131
132     <div class="sec">
133       <h2 class="sec-h">Held-out performance</h2>
134       <dl class="figures" id="figures"></dl>
135     </div>

```

```

136
137 <div class="sec">
138   <h2 class="sec-h">How to read that</h2>
139   <div class="panel">
140     <p id="limits-prose" class="prose"></p>
141     <p class="prose">The split is by patient, never by recording. Each patient contributes
142       several recordings from different auscultation points, so splitting by recording would
143       put the same heart on both sides of the wall and the held-out numbers would be
144       flattering nonsense.</p>
145     <p class="prose">The threshold was chosen on a separate validation split and then
146       applied unchanged to the test set. It sits where it does because a missed murmur sends
147       a child home and an over-referral spends an echo slot. Those costs are not equal.</p>
148   </div>
149 </div>
150
151 <div class="sec">
152   <h2 class="sec-h">Data and credit</h2>
153   <div class="panel">
154     <p class="prose">Trained on the <strong>CirCor DigiScope Phonocardiogram
155       Dataset</strong>, distributed by PhysioNet under the Open Data Commons Attribution
156       Licence (ODC-By 1.0), which requires this attribution.</p>
157     <p class="cite">Oliveira J, Renna F, Costa PD, Nogueira M, Oliveira C, Ferreira C,
158       Jorge A, Mattos S, Hatem T, Tavares T, Elola A, Rad AB, Sameni R, Clifford GD,
159       Coimbra MT. The CirCor DigiScope Dataset: From Murmur Detection to Murmur
160       Classification. <em>IEEE Journal of Biomedical and Health Informatics</em>, 2022.</p>
161     <p class="cite">Goldberger AL, Amaral LAN, Glass L, Hausdorff JM, Ivanov PCh, Mark RG,
162       Mietus JE, Moody GB, Peng C-K, Stanley HE. PhysioBank, PhysioToolkit, and PhysioNet.
163       <em>Circulation</em> 101(23):e215-e220, 2000.</p>
164   </div>
165 </div>
166 </section>
167
168 <section id="view-about" class="view" hidden>
169   <div class="page-head">
170     <h1 class="page-title">How it works</h1>
171     <p class="page-lede">Four stages, no hidden ones. A recording goes in, a single number
172       comes out, and that number is compared against one threshold that was chosen
173       deliberately.</p>
174   </div>
175
176   <div class="sec">
177     <h2 class="sec-h">The pipeline</h2>
178     <ol class="stages" id="stages"></ol>
179   </div>
180 </section>
181
182 <section id="view-limits" class="view" hidden>
183   <div class="page-head">
184     <h1 class="page-title">What this cannot tell you</h1>
185     <p class="page-lede">The honest boundary of the tool, written down rather than left for
186       somebody to discover.</p>
187   </div>
188
189   <div class="sec">
190     <h2 class="sec-h">Limits</h2>
191     <ul class="bullets">
192       <li><strong>A murmur is not rheumatic heart disease.</strong> It is one audible sign
193         that something may be worth imaging. Many children have innocent flow murmurs and
194         perfectly normal valves.</li>
195       <li><strong>Silence is not a clean bill of health.</strong> Valve damage can be present
196         with no murmur an ear or a microphone can find. That is part of why echocardiography
197         exists, and why listening alone cannot settle the question either way.</li>
198       <li><strong>The score is a model output, not a risk.</strong> A score of 0.90 does not
199         mean a 90 per cent chance of disease. It means the model placed this recording at 0.90
200         on the scale it learned, and the only decision attached to that is where it falls
201         relative to the threshold.</li>
202       <li><strong>The setting is unmeasured beyond the source data.</strong> The model was
203         trained on recordings from a screening campaign in Brazil, captured with a digital
204         stethoscope by trained operators. Performance on phone microphones, in noisier rooms,
205         with other populations, has not been measured. Nobody should assume it carries
206         over.</li>
207       <li><strong>Recordings the annotators could not call were excluded.</strong> Human
208         experts labelled some recordings Unknown because they could not hear the answer. Those
209         were kept out of training, and the model has no way to say "I cannot tell" in their
210         place.</li>

```

```

211     </ul>
212 </div>
213
214 <div class="sec">
215     <h2 class="sec-h">Finished and not finished</h2>
216     <ul class="bullets">
217         <li><strong>Finished.</strong> The model, the held-out evaluation, the browser feature
218         pipeline (checked cell by cell against the Python that produced the measured numbers),
219         and this interface. Inference runs entirely on the device.</li>
220         <li><strong>Not finished.</strong> Any validation outside the source dataset. No
221         prospective use, no phone-microphone evaluation, no clinical sign-off. This is a
222         demonstration of a method, not an instrument anyone should screen a child with.</li>
223     </ul>
224 </div>
225 </section>
226
227 <footer class="foot">
228     <p>Murmur runs entirely in the browser. No recording is uploaded, stored, or
229     transmitted.</p>
230     <p class="foot-dim">Not a medical device. Not for clinical use.</p>
231 </footer>
232
233 </main>
234 </div>
235 </div>
236
237 <script type="module" src="app.js"></script>
238 </body>
239 </html>

```

```

1  /* Muxmur -- application chrome.
2  *
3  * A fixed left rail, a top bar over the working column, and content in cards.
4  * Teal is spent only where a decision is: the primary button, the active nav
5  * row, the brand tile, the score marker. Everything else is ink and grey.
6  */
7
8  :root {
9    --ink: #0f2b2e;
10   --teal: #0d7a72;
11   --teal-dark: #0a5f59;
12   --teal-tint: #eaf4f2;
13   --page: #f5f8f7;
14   --card: #ffffff;
15   --hair: #dfe6e5;
16   --grey: #5c6b6c;
17   --rail-w: 250px;
18   --mono: "IBM Plex Mono", ui-monospace, SFMono-Regular, Menlo, monospace;
19   --sans: Inter, -apple-system, BlinkMacSystemFont, "Segoe UI", Helvetica, Arial, sans-serif;
20 }
21
22 * { box-sizing: border-box; }
23
24 html { -webkit-text-size-adjust: 100%; }
25
26 body {
27   margin: 0;
28   background: var(--page);
29   color: var(--ink);
30   font-family: var(--sans);
31   font-size: 15px;
32   line-height: 1.55;
33   overflow-x: hidden;
34 }
35
36 button { font: inherit; color: inherit; }
37
38 .skip {
39   position: absolute;
40   left: -9999px;
41   top: 0;
42   z-index: 50;
43   background: var(--ink);
44   color: #fff;
45   padding: 10px 14px;
46 }
47 .skip:focus { left: 8px; top: 8px; }
48
49 :focus-visible { outline: 2px solid var(--teal); outline-offset: 2px; }
50
51 /* ---- shell ----- */
52
53 .app { display: flex; min-height: 100vh; }
54
55 .rail {
56   position: fixed;
57   top: 0;
58   bottom: 0;
59   left: 0;
60   width: var(--rail-w);
61   display: flex;
62   flex-direction: column;
63   justify-content: space-between;
64   gap: 24px;
65   padding: 18px 14px 16px;
66   background: var(--card);
67   border-right: 1px solid var(--hair);
68   overflow-y: auto;
69   z-index: 10;
70 }
71
72 .main {

```

```

73     flex: 1 1 auto;
74     min-width: 0;
75     margin-left: var(--rail-w);
76     display: flex;
77     flex-direction: column;
78 }
79
80 /* ---- rail ----- */
81
82 .brand {
83     display: flex;
84     align-items: center;
85     gap: 9px;
86     padding: 4px 6px 0;
87     margin-bottom: 16px;
88 }
89
90 .brand-tile {
91     display: inline-flex;
92     align-items: center;
93     justify-content: center;
94     width: 26px;
95     height: 26px;
96     border-radius: 7px;
97     background: var(--teal);
98     color: #fff;
99     flex: none;
100 }
101
102 .brand-name {
103     font-size: 17px;
104     font-weight: 600;
105     letter-spacing: -0.01em;
106 }
107
108 .btn {
109     display: inline-flex;
110     align-items: center;
111     justify-content: center;
112     gap: 8px;
113     border: 1px solid var(--hair);
114     background: var(--card);
115     border-radius: 8px;
116     padding: 9px 16px;
117     font-size: 14px;
118     font-weight: 500;
119     cursor: pointer;
120     text-align: center;
121 }
122 .btn:hover { border-color: #c6d2d1; }
123 .btn[disabled] { opacity: 0.45; cursor: default; }
124
125 .btn-primary {
126     background: var(--teal);
127     border-color: var(--teal);
128     color: #fff;
129     font-weight: 600;
130 }
131 .btn-primary:hover { background: var(--teal-dark); border-color: var(--teal-dark); }
132 .btn-primary[disabled]:hover { background: var(--teal); border-color: var(--teal); }
133
134 .btn-rec { background: var(--ink); border-color: var(--ink); color: #fff; }
135 .btn-rec:hover { background: #0a2023; border-color: #0a2023; }
136
137 .rail-cta { width: 100%; padding: 10px 16px; }
138
139 .nav {
140     display: flex;
141     flex-direction: column;
142     gap: 1px;
143     margin-top: 18px;
144 }
145
146 .nav-item {
147     display: flex;

```

```

148     align-items: center;
149     gap: 11px;
150     width: 100%;
151     padding: 8px 10px;
152     border: 0;
153     border-radius: 7px;
154     background: none;
155     color: var(--ink);
156     font-size: 14px;
157     font-weight: 500;
158     cursor: pointer;
159     text-align: left;
160 }
161 .nav-item:hover { background: #f1f5f4; }
162 .nav-item .nav-ico { color: var(--grey); flex: none; }
163 .nav-item.is-on { background: var(--teal-tint); color: var(--teal-dark); }
164 .nav-item.is-on .nav-ico { color: var(--teal); }
165
166 .rail-foot { border-top: 1px solid var(--hair); padding: 14px 8px 0; }
167
168 .usage + .usage { margin-top: 12px; }
169
170 .usage-row {
171     display: flex;
172     justify-content: space-between;
173     align-items: baseline;
174     font-size: 12.5px;
175     color: var(--grey);
176 }
177 .usage-row .num { font-family: var(--mono); font-size: 12px; color: var(--ink); }
178
179 .usage-bar {
180     height: 3px;
181     margin-top: 6px;
182     border-radius: 2px;
183     background: #eaeefee;
184     overflow: hidden;
185 }
186 .usage-bar span { display: block; height: 100%; background: var(--teal); }
187
188 .rail-note {
189     margin: 14px 0 0;
190     font-size: 11.5px;
191     line-height: 1.5;
192     color: var(--grey);
193 }
194
195 /* ---- top bar ----- */
196
197 .topbar {
198     position: sticky;
199     top: 0;
200     z-index: 6;
201     display: grid;
202     grid-template-columns: 1fr minmax(0, 600px) 1fr;
203     align-items: center;
204     gap: 16px;
205     padding: 11px 28px;
206     background: var(--card);
207     border-bottom: 1px solid var(--hair);
208 }
209
210 .topbar-field {
211     grid-column: 2;
212     display: flex;
213     align-items: center;
214     gap: 9px;
215     min-width: 0;
216     padding: 8px 14px;
217     border: 1px solid var(--hair);
218     border-radius: 9px;
219     background: var(--page);
220     color: var(--grey);
221     font-size: 13.5px;
222 }

```

```

223 .topbar-field span {
224     overflow: hidden;
225     text-overflow: ellipsis;
226     white-space: nowrap;
227 }
228 .topbar-field svg { flex: none; }
229
230 .topbar-acts {
231     grid-column: 3;
232     justify-self: end;
233     display: flex;
234     align-items: center;
235     gap: 18px;
236 }
237 .topbar-act { font-size: 13px; color: var(--grey); white-space: nowrap; }
238
239 /* ---- content ----- */
240
241 .content {
242     flex: 1 1 auto;
243     width: 100%;
244     max-width: 1140px;
245     margin: 0 auto;
246     padding: 40px 28px 56px;
247 }
248
249 .page-head { max-width: 700px; margin: 0 auto 40px; text-align: center; }
250
251 .page-title {
252     margin: 0;
253     font-size: 34px;
254     line-height: 1.18;
255     font-weight: 600;
256     letter-spacing: -0.022em;
257 }
258
259 .page-lede { margin: 14px 0 0; color: var(--grey); font-size: 14.5px; }
260
261 .sec { margin-bottom: 36px; }
262
263 .sec-h {
264     margin: 0 0 4px;
265     font-size: 17px;
266     font-weight: 600;
267     letter-spacing: -0.01em;
268 }
269
270 .sec-note { margin: 0 0 14px; color: var(--grey); font-size: 13.5px; max-width: 68ch; }
271
272 .boot {
273     margin: 0 0 26px;
274     padding: 10px 14px;
275     border: 1px solid var(--hair);
276     border-radius: 8px;
277     background: var(--card);
278     color: var(--grey);
279     font-size: 13.5px;
280 }
281 .boot[hidden] { display: none; }
282
283 /* ---- cards ----- */
284
285 .cardrow {
286     display: grid;
287     grid-template-columns: repeat(auto-fit, minmax(196px, 1fr));
288     gap: 12px;
289 }
290
291 .card {
292     display: block;
293     width: 100%;
294     padding: 14px;
295     border: 1px solid var(--hair);
296     border-radius: 10px;
297     background: var(--card);

```

```

298   text-align: left;
299 }
300
301 button.card { cursor: pointer; font: inherit; color: inherit; }
302 button.card:hover { border-color: #bfcecd; }
303 button.card[disabled] { opacity: 0.5; cursor: default; }
304 button.card[disabled]:hover { border-color: var(--hair); }
305 button.card[aria-pressed="true"] { border-color: var(--teal); box-shadow: inset 0 0 1px var(--teal); }
306
307 .card-head { display: flex; align-items: flex-start; gap: 11px; }
308
309 .tile {
310   display: inline-flex;
311   align-items: center;
312   justify-content: center;
313   width: 32px;
314   height: 32px;
315   flex: none;
316   border-radius: 8px;
317   background: var(--teal-tint);
318   color: var(--teal);
319 }
320
321 .card-t { font-size: 14.5px; font-weight: 600; letter-spacing: -0.005em; }
322 .card-s { margin-top: 1px; font-size: 12.5px; color: var(--grey); }
323
324 .card-wide { max-width: 720px; }
325 .card-body { margin: 14px 0 0; font-size: 13.5px; color: var(--grey); }
326
327 .card-warn {
328   margin: 14px 0 0;
329   padding-top: 12px;
330   border-top: 1px solid var(--hair);
331   font-size: 12.5px;
332   color: var(--grey);
333 }
334
335 .mic { display: flex; align-items: center; gap: 14px; flex-wrap: wrap; margin-top: 14px; }
336 .mic-state { font-family: var(--mono); font-size: 12.5px; color: var(--grey); }
337
338 .meter {
339   width: 100%;
340   height: 4px;
341   border-radius: 2px;
342   background: #eaefee;
343   overflow: hidden;
344 }
345 .meter[hidden] { display: none; }
346 .meter-fill { display: block; height: 100%; width: 0; background: var(--teal); }
347
348 /* ---- status ----- */
349
350 .status, .error {
351   margin: 0 0 20px;
352   padding: 10px 14px;
353   border: 1px solid var(--hair);
354   border-radius: 8px;
355   background: var(--card);
356   font-size: 13.5px;
357   color: var(--grey);
358 }
359 .status[hidden], .error[hidden] { display: none; }
360 .error { border-color: #d9b4b0; color: #8c2f26; background: #fdf4f3; }
361
362 /* ---- result panel ----- */
363
364 .result {
365   border: 1px solid var(--hair);
366   border-radius: 12px;
367   background: var(--card);
368   overflow: hidden;
369 }
370 .result[hidden] { display: none; }
371
372 .result-head { padding: 26px 26px 22px; border-bottom: 1px solid var(--hair); }

```

```

373
374 .result-src {
375     margin: 0 0 8px;
376     font-family: var(--mono);
377     font-size: 12px;
378     color: var(--grey);
379 }
380
381 .verdict {
382     margin: 0;
383     font-size: 25px;
384     font-weight: 600;
385     letter-spacing: -0.02em;
386 }
387 .verdict.is-refer { color: var(--teal-dark); }
388
389 .verdict-sub { margin: 8px 0 0; max-width: 72ch; color: var(--grey); font-size: 14px; }
390
391 .block { padding: 24px 26px; border-top: 1px solid var(--hair); }
392 .block:first-child { border-top: 0; }
393
394 .block-h {
395     margin: 0 0 16px;
396     font-size: 14px;
397     font-weight: 600;
398 }
399
400 /* ---- score scale ----- */
401
402 .scale { position: relative; margin: 30px 0 0; }
403
404 .scale-top { position: relative; height: 40px; }
405
406 .scale-score {
407     position: absolute;
408     bottom: 0;
409     white-space: nowrap;
410     line-height: 1.25;
411 }
412 .scale-score b {
413     display: block;
414     font-family: var(--mono);
415     font-size: 18px;
416     font-weight: 500;
417     color: var(--teal-dark);
418 }
419 .scale-score span { font-size: 11.5px; color: var(--grey); }
420
421 .scale-mark { position: relative; height: 7px; }
422 .scale-tri {
423     position: absolute;
424     bottom: 0;
425     width: 0;
426     height: 0;
427     margin-left: -5px;
428     border-left: 5px solid transparent;
429     border-right: 5px solid transparent;
430     border-top: 6px solid var(--teal);
431 }
432
433 .scale-track {
434     position: relative;
435     height: 12px;
436     border-radius: 6px;
437     background: #eef2f1;
438     border: 1px solid var(--hair);
439 }
440
441 .scale-over {
442     position: absolute;
443     top: 0;
444     bottom: 0;
445     right: 0;
446     background: #d3e8e4;
447     border-radius: 0 5px 5px 0;

```

```

448 }
449
450 .scale-thr {
451   position: absolute;
452   top: -4px;
453   bottom: -4px;
454   width: 1px;
455   background: var(--ink);
456 }
457
458 .scale-pin {
459   position: absolute;
460   top: 50%;
461   width: 12px;
462   height: 12px;
463   margin: -6px 0 0 -6px;
464   border-radius: 50%;
465   background: var(--teal);
466   border: 2px solid var(--card);
467 }
468
469 .scale-ends {
470   display: flex;
471   justify-content: space-between;
472   margin-top: 6px;
473   font-family: var(--mono);
474   font-size: 11px;
475   color: var(--grey);
476 }
477
478 .scale-labels { position: relative; height: 34px; }
479
480 .scale-tag { position: absolute; top: 0; white-space: nowrap; line-height: 1.3; }
481 .scale-tag b {
482   display: block;
483   font-family: var(--mono);
484   font-size: 12px;
485   font-weight: 500;
486 }
487 .scale-tag span { font-size: 11.5px; color: var(--grey); }
488
489 .scale-note {
490   margin: 8px 0 0;
491   max-width: 74ch;
492   font-size: 13.5px;
493   color: var(--grey);
494 }
495 .scale-gap { font-family: var(--mono); color: var(--ink); }
496
497 /* ---- segments ----- */
498
499 .note { margin: 0 0 16px; max-width: 74ch; font-size: 13.5px; color: var(--grey); }
500
501 .segs {
502   position: relative;
503   display: flex;
504   align-items: flex-end;
505   gap: 10px;
506   height: 140px;
507   margin-top: 22px;
508   padding: 0 2px;
509   border-bottom: 1px solid var(--hair);
510 }
511
512 .seg-thr {
513   position: absolute;
514   left: 0;
515   right: 0;
516   height: 0;
517   border-top: 1px dashed #9fb0b0;
518   pointer-events: none;
519 }
520 .seg-thr-tag {
521   position: absolute;
522   right: 0;

```

```

523   top: -15px;
524   font-family: var(--mono);
525   font-size: 10px;
526   color: var(--grey);
527   background: var(--card);
528   padding: 0 3px;
529 }
530
531 .seg {
532   position: relative;
533   flex: 1 1 0;
534   max-width: 88px;
535   height: 100%;
536   display: flex;
537   align-items: flex-end;
538   padding: 0;
539   border: 0;
540   background: none;
541   cursor: pointer;
542 }
543
544 .seg-bar {
545   display: block;
546   width: 100%;
547   background: #cbd8d7;
548   border-radius: 3px 3px 0 0;
549 }
550 .seg:hover .seg-bar { background: #b3c4c3; }
551 .seg.is-top .seg-bar { background: var(--teal); }
552 .seg[aria-pressed="true"] .seg-bar { outline: 2px solid var(--ink); outline-offset: -2px; }
553
554 .seg-axis {
555   display: flex;
556   gap: 10px;
557   padding: 0 2px;
558   margin-top: 6px;
559 }
560 .seg-axis span {
561   flex: 1 1 0;
562   max-width: 88px;
563   text-align: center;
564   font-family: var(--mono);
565   font-size: 10.5px;
566   color: var(--grey);
567 }
568 .seg-axis span.is-sel { color: var(--ink); }
569
570 .seg-pick {
571   display: flex;
572   align-items: center;
573   justify-content: space-between;
574   gap: 16px;
575   flex-wrap: wrap;
576   margin-top: 18px;
577   padding: 12px 14px;
578   border: 1px solid var(--hair);
579   border-radius: 9px;
580   background: var(--page);
581 }
582
583 .seg-facts { font-size: 13px; color: var(--grey); }
584 .seg-facts b {
585   display: block;
586   font-family: var(--mono);
587   font-size: 12.5px;
588   font-weight: 500;
589   color: var(--ink);
590   white-space: pre;
591 }
592
593 /* ---- reference annotation ----- */
594
595 .truth-row {
596   display: grid;
597   grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));

```

```

598     gap: 8px;
599     margin: 0 0 16px;
600 }
601 .truth-cell {
602     padding: 11px 13px;
603     border: 1px solid var(--hair);
604     border-radius: 8px;
605     background: var(--card);
606 }
607 .truth-cell dt { font-size: 11.5px; color: var(--grey); }
608 .truth-cell dd {
609     margin: 2px 0 0;
610     font-family: var(--mono);
611     font-size: 13px;
612 }
613
614 .truth-call {
615     margin: 0;
616     padding: 13px 15px;
617     border: 1px solid var(--hair);
618     border-left: 3px solid var(--grey);
619     border-radius: 0 9px 9px 0;
620     background: var(--page);
621     font-size: 13.5px;
622     color: var(--grey);
623     max-width: 80ch;
624 }
625 .truth-call b { color: var(--ink); }
626 .truth-call.call-hit { border-left-color: var(--teal); }
627 .truth-call.call-miss { border-left-color: #8c2f26; }
628 .truth-call.call-over { border-left-color: #9a6b1f; }
629
630 /* ---- how it works ----- */
631
632 .panel {
633     max-width: 860px;
634     padding: 20px 22px;
635     border: 1px solid var(--hair);
636     border-radius: 10px;
637     background: var(--card);
638 }
639
640 .stages { list-style: none; margin: 0; padding: 0; display: grid; gap: 12px; max-width: 880px; }
641
642 .stage {
643     display: flex;
644     gap: 16px;
645     padding: 18px 20px;
646     border: 1px solid var(--hair);
647     border-radius: 10px;
648     background: var(--card);
649 }
650
651 .stage-n {
652     flex: none;
653     font-family: var(--mono);
654     font-size: 12px;
655     color: var(--teal);
656     padding-top: 2px;
657 }
658
659 .stage-t { font-size: 14.5px; font-weight: 600; }
660 .stage-d { margin-top: 5px; font-size: 13.5px; color: var(--grey); max-width: 72ch; }
661 .stage-k {
662     margin-top: 8px;
663     font-family: var(--mono);
664     font-size: 11.5px;
665     color: var(--grey);
666 }
667
668 .figures {
669     display: grid;
670     grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
671     gap: 12px;
672     margin: 0;

```

```

673 }
674 .figures > div {
675   padding: 16px 18px;
676   border: 1px solid var(--hair);
677   border-radius: 10px;
678   background: var(--card);
679 }
680 .figures dt { font-size: 12.5px; color: var(--grey); }
681 .figures dd {
682   margin: 6px 0 0;
683   font-family: var(--mono);
684   font-size: 24px;
685   line-height: 1.1;
686 }
687 .fig-sub {
688   display: block;
689   margin-top: 6px;
690   font-family: var(--sans);
691   font-size: 12px;
692   color: var(--grey);
693 }
694
695 .prose { margin: 0 0 14px; font-size: 14px; max-width: 78ch; }
696 .prose:last-child { margin-bottom: 0; }
697
698 .cite {
699   margin: 0 0 12px;
700   font-size: 12px;
701   line-height: 1.6;
702   color: var(--grey);
703   max-width: 88ch;
704 }
705 .cite:last-child { margin-bottom: 0; }
706
707 .bullets { list-style: none; margin: 0; padding: 0; display: grid; gap: 10px; max-width: 880px; }
708 .bullets li {
709   padding: 16px 18px;
710   border: 1px solid var(--hair);
711   border-radius: 10px;
712   background: var(--card);
713   font-size: 13.5px;
714   color: var(--grey);
715   max-width: 82ch;
716 }
717 .bullets strong { color: var(--ink); }
718
719 /* ---- footer ----- */
720
721 .foot {
722   margin-top: 48px;
723   padding-top: 18px;
724   border-top: 1px solid var(--hair);
725   font-size: 12.5px;
726   color: var(--grey);
727 }
728 .foot p { margin: 0 0 4px; }
729 .foot-dim { color: #8b9797; }
730
731 /* ---- narrow ----- */
732
733 @media (max-width: 900px) {
734   .app { flex-direction: column; }
735
736   .rail {
737     position: static;
738     width: auto;
739     flex-direction: row;
740     align-items: center;
741     justify-content: flex-start;
742     gap: 16px;
743     flex-wrap: wrap;
744     padding: 12px 18px;
745     border-right: 0;
746     border-bottom: 1px solid var(--hair);
747     overflow: visible;

```

```

748 }
749
750 .rail-top {
751     display: flex;
752     align-items: center;
753     gap: 16px;
754     flex-wrap: wrap;
755     flex: 1 1 auto;
756     min-width: 0;
757 }
758
759 .brand { margin: 0; padding: 0; }
760 .rail-cta { width: auto; }
761 .nav { flex-direction: row; margin: 0; gap: 2px; flex-wrap: wrap; }
762
763 .rail-foot {
764     display: flex;
765     align-items: center;
766     gap: 20px;
767     border-top: 0;
768     padding: 0;
769 }
770 .usage + .usage { margin-top: 0; }
771 .usage { min-width: 104px; }
772 .rail-note { display: none; }
773
774 .main { margin-left: 0; }
775 .topbar { padding: 10px 18px; grid-template-columns: minmax(0, 1fr) auto; }
776 .topbar-field { grid-column: 1; }
777 .topbar-acts { grid-column: 2; }
778 .content { padding: 30px 18px 44px; }
779 .page-title { font-size: 28px; }
780 }
781
782 @media (max-width: 640px) {
783     .topbar { grid-template-columns: minmax(0, 1fr); }
784     .topbar-acts { grid-column: 1; justify-self: start; gap: 14px; }
785     .topbar-act { font-size: 12px; }
786     .rail-foot { display: none; }
787     .content { padding: 24px 16px 40px; }
788     .page-title { font-size: 24px; }
789     .verdict { font-size: 21px; }
790     .result-head { padding: 20px 16px 18px; }
791     .block { padding: 20px 16px; }
792     .cardrow { grid-template-columns: 1fr; }
793     .seg-facts b { white-space: normal; }
794 }

```